MICROCONTROLLER BASED SYSTEMS & ROBOTICS HUB

Electronics, Arduino & ESP32

From the breadboard to your first sensor-driven sketch: circuits, microcontrollers, actuators and programming.



ROADMAP

Four parts, one build path

01

Electronics

Breadboard, current, voltage, resistors, LDR, diode, LED, capacitor, switches

02

Controllers

Microcontroller vs microprocessor, robot parts, ESP family, ESP32 vs Arduino

03

Actuators

Actuator types, DC motors, brushed vs brushless, H-bridge, servo

04

Programming

Functions, scope, Serial, operators, if-else, switch, loops, sensors

ELECTRONICS BASICS

The breadboard

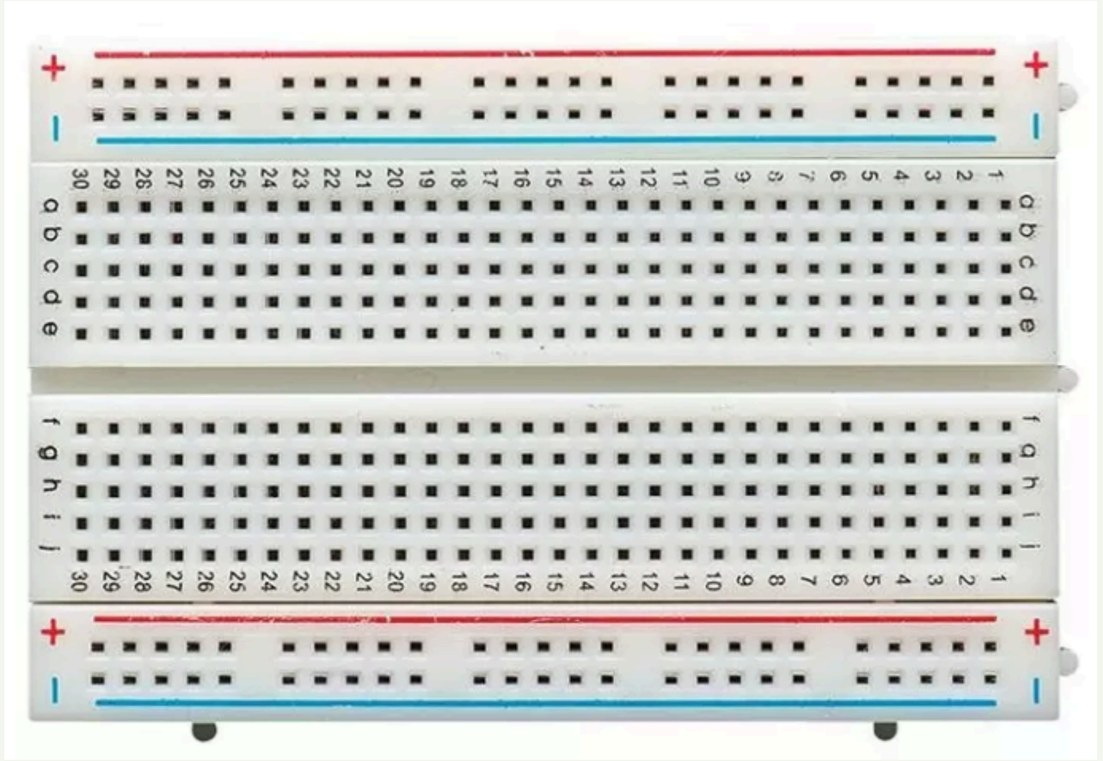
A reusable, solder-free board for building and testing circuits. Push in parts and wires; pull them out to change the design.

Strips: each column of 5 holes is joined underneath.

Centre gap: splits the two halves; ICs sit across it.

Power rails: long + and – lines run the full length.

Jumper wires: link strips, rails and the Arduino.



ELECTRONICS BASICS

Conductors and insulators

Conductors

Atoms hold their outer electrons loosely, so charge flows easily.

Copper, silver, gold, aluminium, iron, salt water

Used for: wires, PCB tracks, jumper leads

Insulators

Electrons are tightly bound, so almost no current flows.

Rubber, plastic, glass, wood, dry air

Used for: wire coating, enclosures, safety

In between: semiconductors such as silicon conduct only under the right conditions. Diodes, LEDs and transistors are built from them.

ELECTRONICS BASICS

Electric current and how it flows

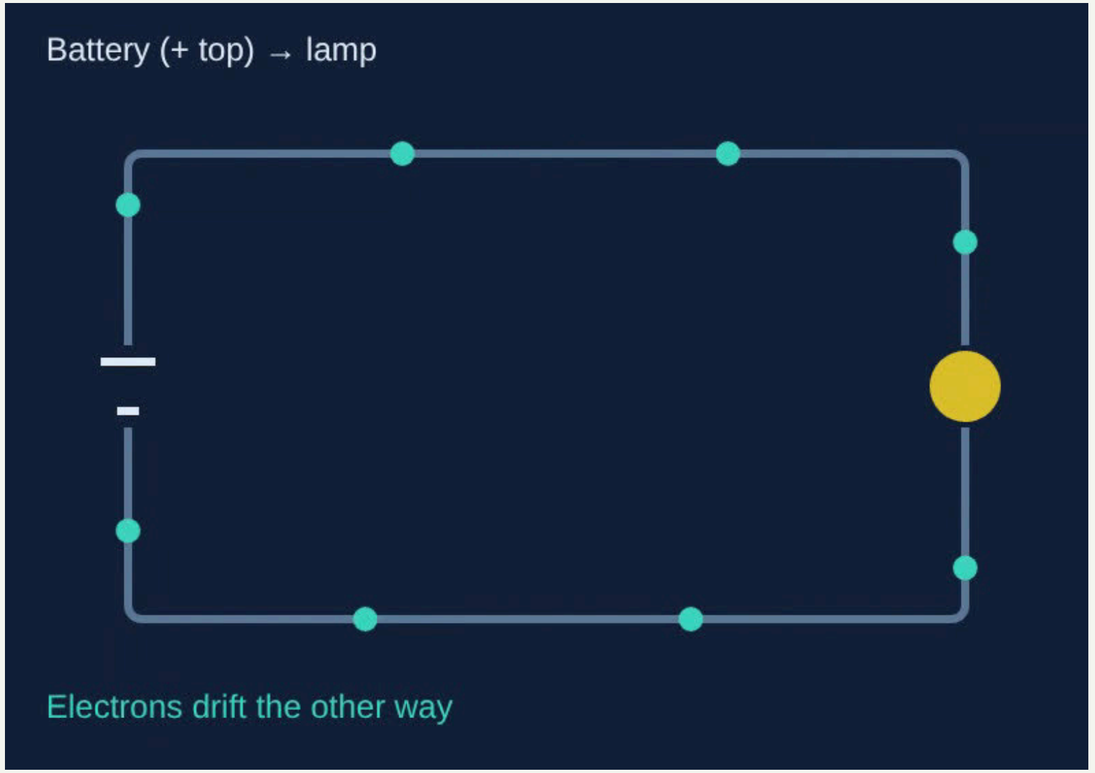
Current is the flow of electric charge. It is measured in amperes (A): coulombs of charge passing per second.

$$I = Q \div t$$

Closed loop: current flows only in a complete circuit.

Electrons move from − to +.

Conventional current is drawn + to −.



ELECTRONICS BASICS

Batteries and voltage

Battery

Stores chemical energy and turns it into electrical energy. Two terminals: positive (+) and negative (−).

Cells in series add their voltage.

Voltage (V)

The electrical “push” between two points, also called potential difference. Measured in volts.

Water analogy: voltage is pressure, current is flow.

1.5 V
AA cell

3.7 V
Li-ion cell

9 V
Block battery

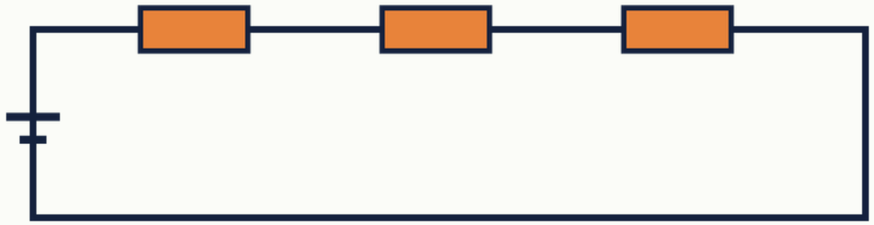
5 V
Arduino Uno logic

3.3 V
ESP32 logic

ELECTRONICS BASICS

Series and parallel circuits

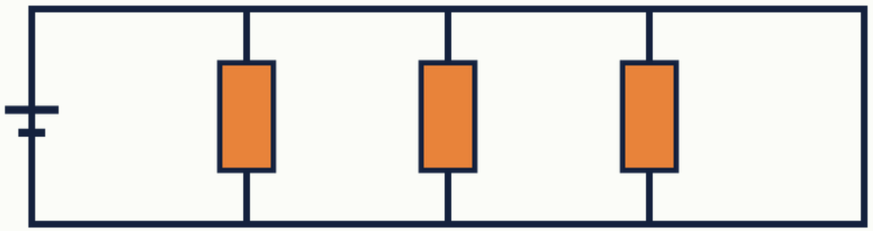
Series: one path



Same current everywhere. Voltages add up. If one part breaks, all stop.

```
Rtotal = R1 + R2 + R3
```

Parallel: many paths



Same voltage on each branch. Currents add up. Others keep working.

```
1/Rtotal = 1/R1 + 1/R2 + 1/R3
```

ELECTRONICS BASICS

Resistance, resistors and Ohm's law

Resistance (R) opposes current. It is measured in ohms (Ω). A resistor adds a known resistance to limit current and protect parts such as LEDs.

$$V = I \times R$$

Voltage = current × resistance

More resistance means less current for the same voltage.

Worked example: LED

Supply 5 V, LED drops about 2 V, resistor 220 Ω.

```
I = (5 - 2) ÷ 220
I ≈ 0.0136 A
I ≈ 13.6 mA
```

Safe for a standard LED (about 20 mA max).

ELECTRONICS BASICS

Resistor colour code

0	1	2	3	4	5	6	7	8	9
Black	Brown	Red	Orange	Yellow	Green	Blue	Violet	Grey	White

Read the bands

Band 1 and 2 are digits. Band 3 is the multiplier (× 10 to the power of the colour number). Band 4 is tolerance.

Tolerance

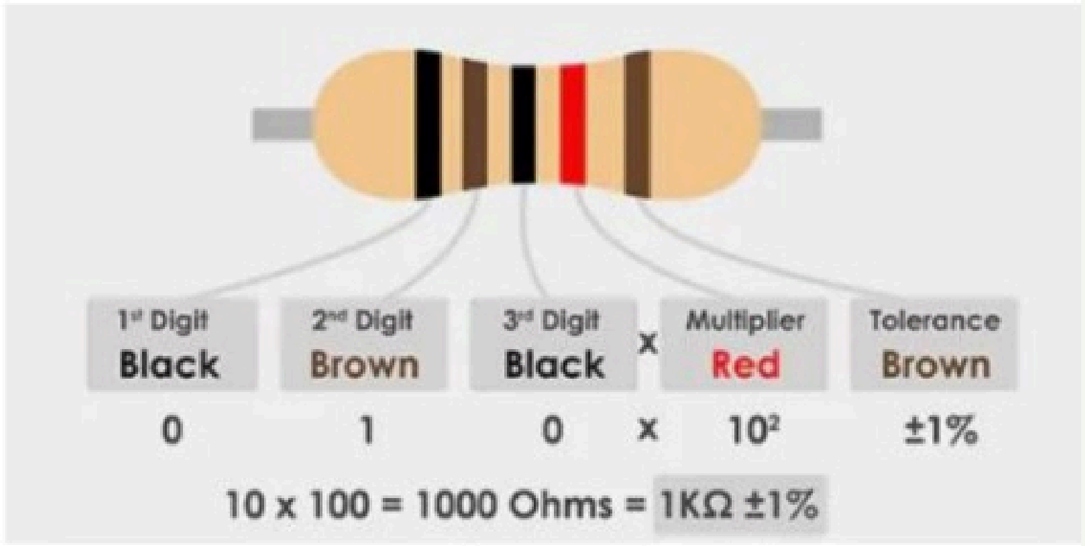
Gold = ±5 %. Silver = ±10 %. No fourth band = ±20 %.

Try it: Red, Violet, Orange, Gold = 2, 7, × 1000 = **27 kΩ ±5 %**

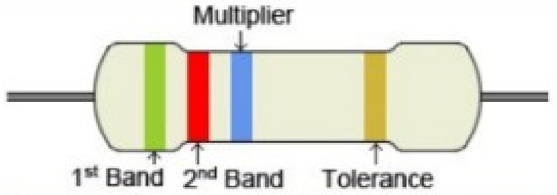
Resistor colour code

0	1	2	3	4	5	6	7	8	9
Black	Brown	Red	Orange	Yellow	Green	Blue	Violet	Grey	White

- The resistance value is calculated as shown



Source



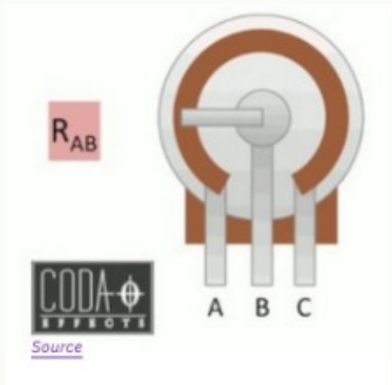
Color	1st, 2nd Band Significant Figures	Multiplier	Tolerance
Black	0	× 1	
Brown	1	× 10	±1% (F)
Red	2	× 100	±2% (G)
Orange	3	× 1K	±0.05% (W)
Yellow	4	× 10K	±0.02% (P)
Green	5	× 100K	±0.5% (D)
Blue	6	× 1M	±0.25% (C)
Violet	7	× 10M	±0.1% (B)
Grey	8	× 100M	±0.01% (L)
White	9	× 1G	
Gold		× 0.1	±5% (J)
Silver		× 0.01	±10% (K)

Components: part 1

Variable resistor

Three pins and a turning knob (wiper). Resistance changes as you turn it.

Potentiometer: volume, dimmers, analog input.



LDR

Light dependent resistor. Resistance falls as light rises.

Used with a fixed resistor to sense brightness for street lights.



Diode

Lets current flow one way only: anode to cathode.

The stripe marks the cathode. Used for rectifying and protection.



LED

A diode that gives light. The longer leg is the anode (+).

Always add a series resistor.



⚠ **Polarity matters:** a diode or LED fitted backwards will not conduct.

ELECTRONICS BASICS

Components: part 2

Capacitor

Stores charge briefly and releases it. Measured in farads (µF, nF, pF).

Electrolytic types are polarised: the longer leg is +.

Uses: smoothing power, filtering noise, timing.



Switches

Open or close a circuit. Types: push button, toggle, slide, DIP.

Normally open (NO) or normally closed (NC).

With Arduino, add a pull-up or pull-down resistor.

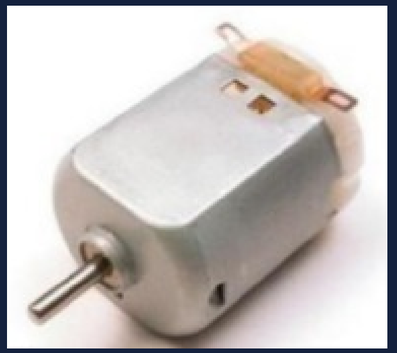


Motor

Turns electrical energy into rotation.

Needs more current than a pin can give, so it is driven through a driver board.

Details in the actuators part.



KNOWLEDGE CHECK 1

Electronics quiz

Q1. Bands: Brown, Black, Red, Gold. What value?

A) 100 Ω ±5 %

B) 1 kΩ ±5 %

C) 10 kΩ ±5 %

D) 102 Ω ±10 %

Q2. In which circuit does one broken lamp turn the others off?

A) Series

B) Parallel

C) Both

D) Neither

KNOWLEDGE CHECK 1

Electronics quiz

Q1. Bands: Brown, Black, Red, Gold. What value?

A) 100 Ω ±5 %

B) 1 kΩ ±5 %

C) 10 kΩ ±5 %

D) 102 Ω ±10 %

Answer: B. 10 × 10^2 = 1000 Ω, tolerance gold = ±5 %.

Q2. In which circuit does one broken lamp turn the others off?

A) Series

B) Parallel

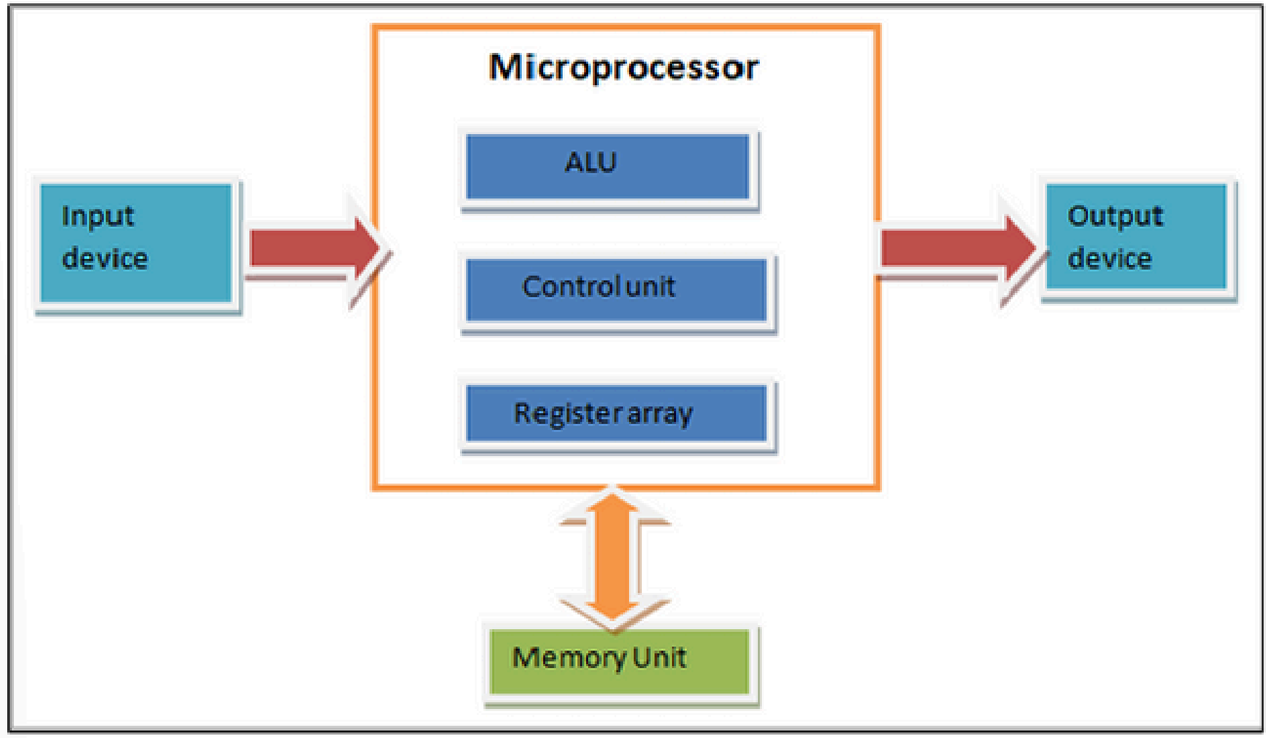
C) Both

D) Neither

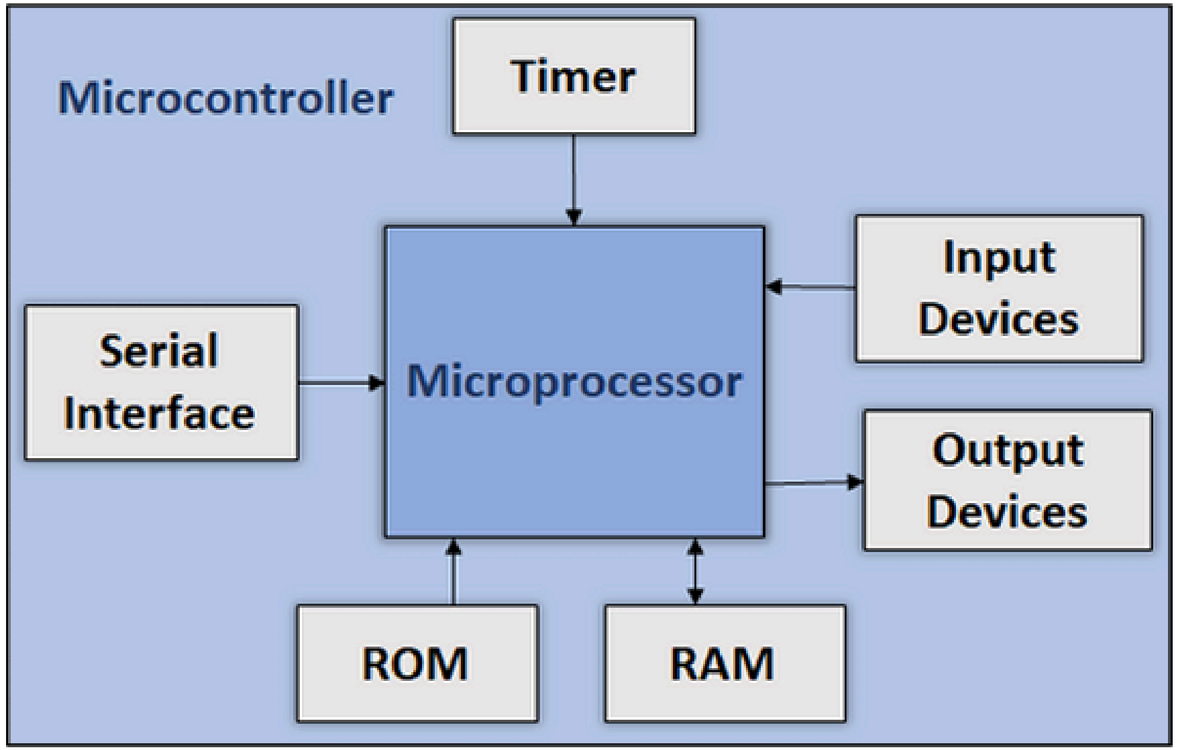
Answer: A. Series has a single path, so one break stops all.

MICROPROCESSOR VS MICROCONTROLLER

- The key difference: Microprocessor consists of only a Central Processing Unit, whereas the microcontroller contains a CPU, memory, I/O all integrated into one chip



Source



Source

MORE ABOUT THE DEVICES

What is the difference between a computer, a microprocessor and a microcontroller ?

- Microprocessor is a full computation engine fabricated on a single chip; It acts as the central processing unit(CPU) of a microcomputer

  A Computer is a microprocessor
- many interfaces and memory chips packaged on a single circuit board with ; General purpose computers, i.e., PCs, are designed explicitly tointerface withhumans

- Microcontrollers are designed to interface, interact, andcommunicate with

➢ Electrical/electronic devices
➢ Sensor/actuators
➢ High-tech gadgets, etc.

MICROCONTROLLERS

Components of a robot

Sensors

Eyes and ears: light, distance, touch, temperature.

Controller

The brain: an Arduino or ESP32 runs your code.

Actuators

Muscles: motors, servos and solenoids make it move.

Power supply

Batteries and regulators feed each part the right voltage.

Chassis

The body and frame that holds wheels, arms and boards.

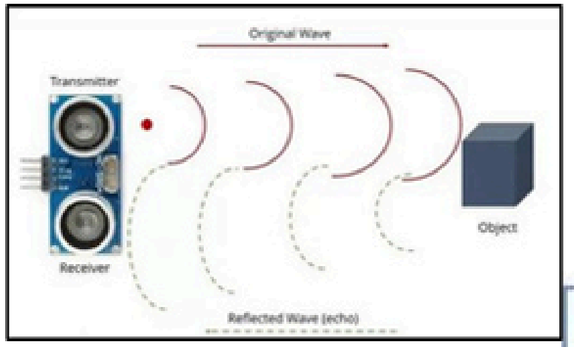
Communication

Wi-Fi, Bluetooth or serial links to a phone or PC.

COMPONENTS OF A ROBOT



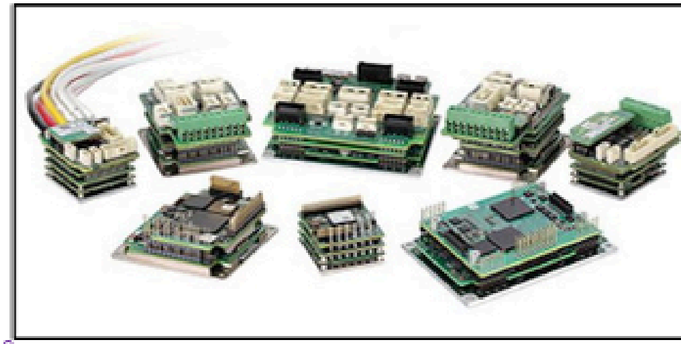
Source



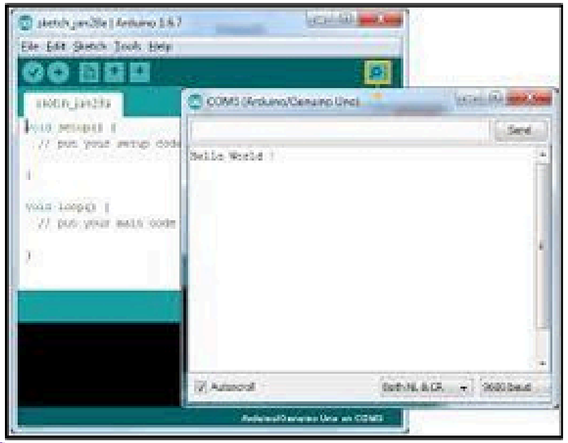
Source



Source



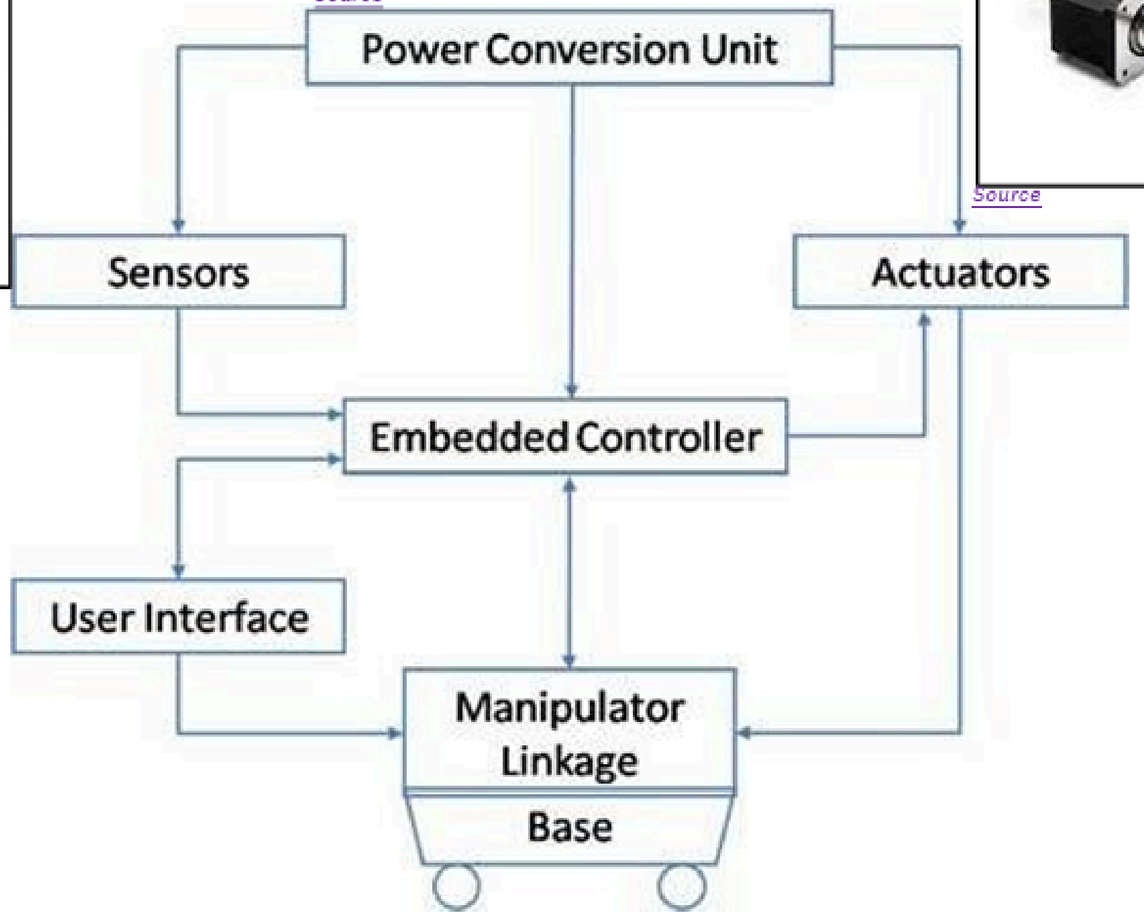
Source



Source

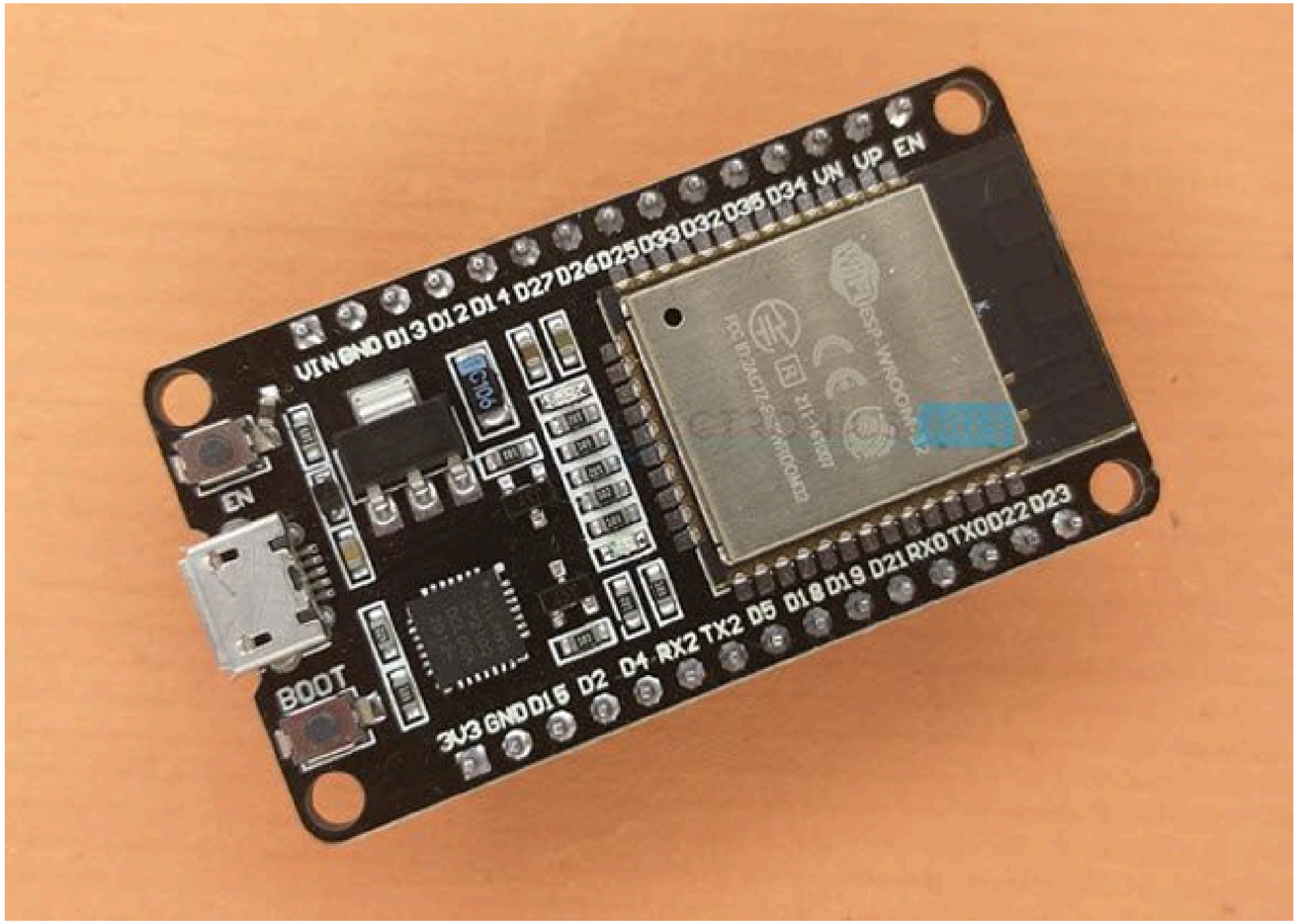


Source



GETTING STARTED WITH ESP32

In this Session, we will learn about ESP32, a dual core MCU from Espressif Systems with integrated Wi-Fi and Bluetooth. If one has worked with ESP8266, then ESP32 is a significant upgrade with a lot more features. This Getting Started with ESP32 guide is for complete beginners, with or without prior experience in IoT or ESP8266.

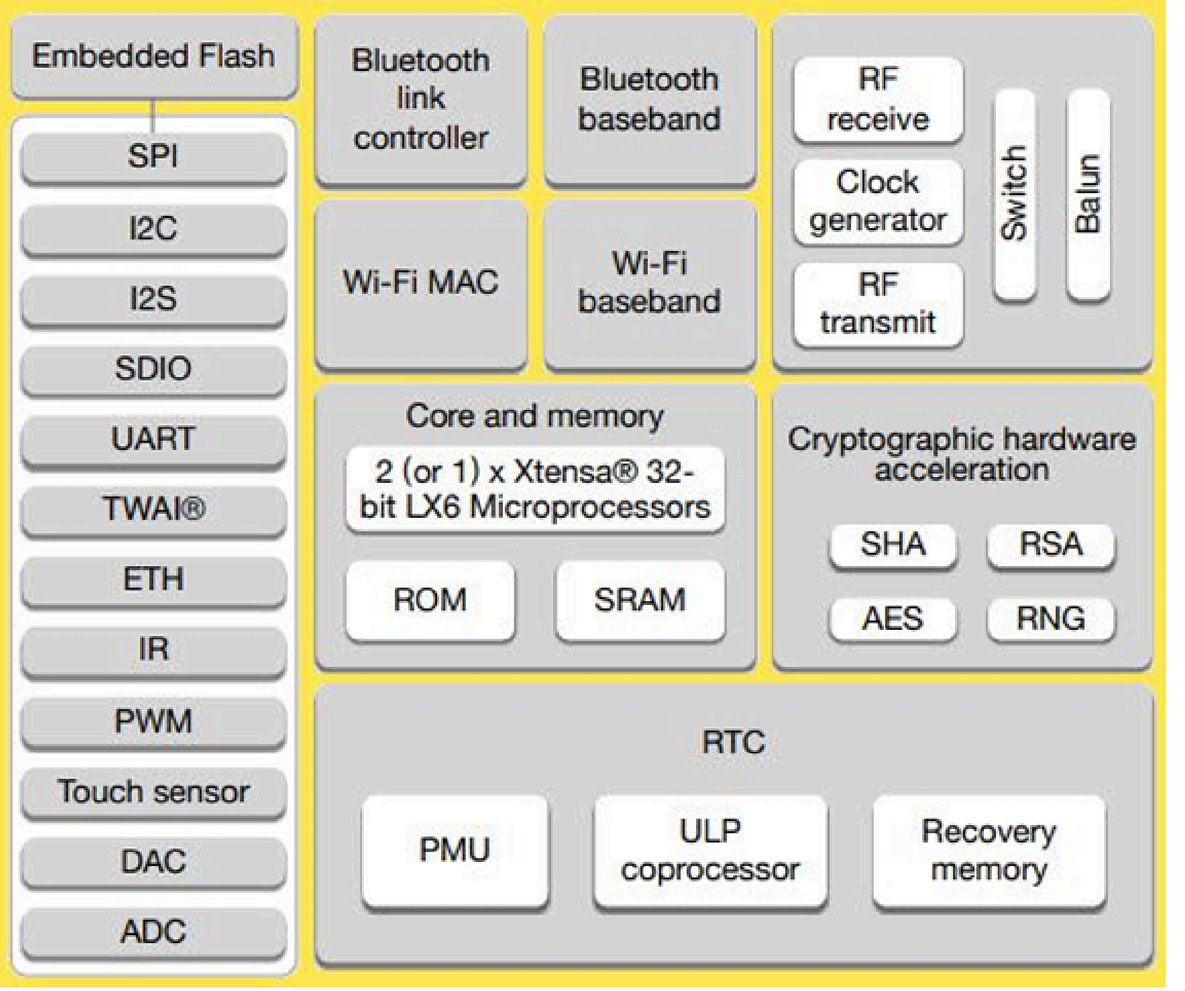


WHAT IS ESP32 ?

- ESP32 is a low-cost System on Chip (SoC) Microcontroller from Espressif Systems, the developers of the famous ESP8266 SoC. It is a successor to ESP8266 SoC and comes in both single-core and dual-core variations of the Tensilica's 32-bit Xtensa LX6 Microprocessor with integrated Wi-Fi and Bluetooth.

- The good thing about ESP32, like ESP8266 is its integrated RF components like Power Amplifier, Low-Noise Receive Amplifier, Antenna Switch, Filters and RF Balun. This makes designing hardware around ESP32 very easy as you require very few external components.

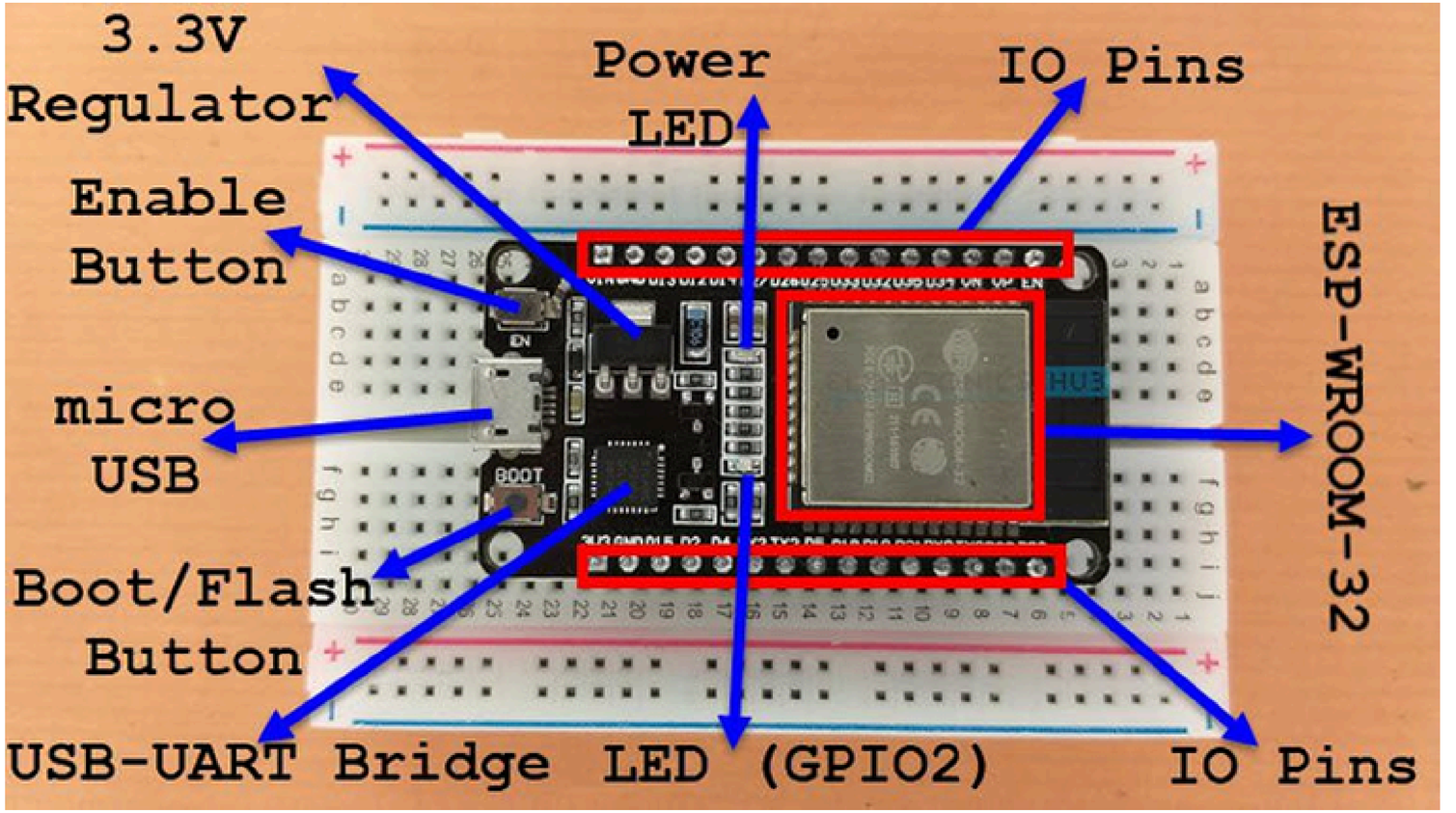
- Another important thing to know about ESP32 is that it is manufactured using TSMC's ultra-low-power 40 nm technology. So, designing battery operated applications like wearables, audio equipment, baby monitors, smart watches, etc., using ESP32 should be very easy.



LAYOUT

As you can see from the image, the ESP32 Board consists of the following:

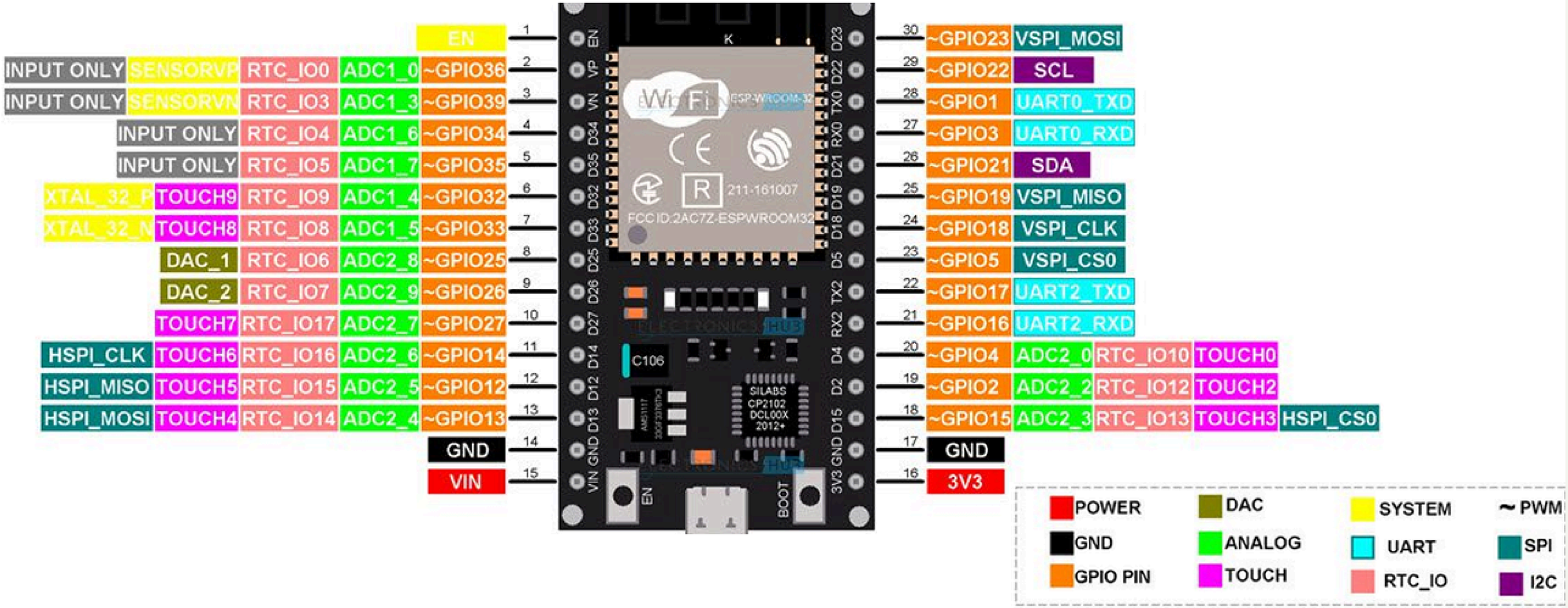
- ESP-WROOM-32 Module
- Two rows of IO Pins
  (with 15 pins on each side)
- CP2012 USB – UART Bridge IC
- micro–USB Connector
  (for power and programming)
- AMS1117 3.3V Regulator IC
- Enable Button (for Reset)
- Boot Button (for flashing)
- Power LED (Red)
- User LED (Blue – connected to GPIO2)
- Some passive components



An interesting point about the USB-to-UART IC is that its DTR and RTS pins are used to automatically set the ESP32 in to programming mode (whenever required) and also rest the board after programming.

PINOUT OF ESP32 BOARD

This is the pinout diagram for the 30 - pin version of the ESP32 Development Board.

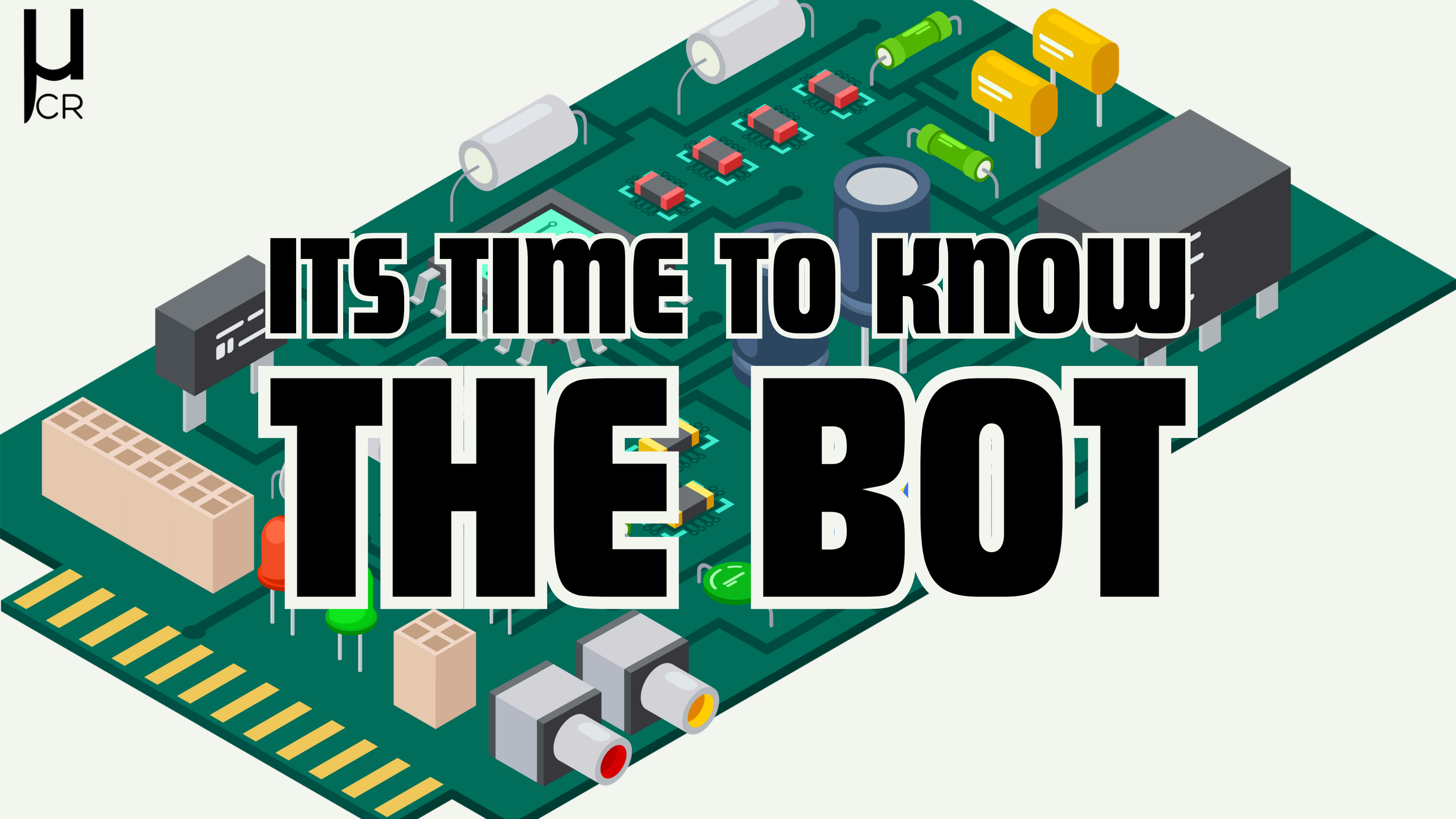


ESP FAMILY

How ESP32 differs from Arduino

Feature	Arduino Uno	ESP32 DevKit
Chip	ATmega328P, 8-bit	Xtensa LX6, 32-bit dual-core
Clock	16 MHz	Up to 240 MHz
Memory	2 KB SRAM, 32 KB flash	520 KB SRAM, 4 MB flash
Logic level	5 V	3.3 V (not 5 V tolerant)
Wireless	None built in	Wi-Fi and Bluetooth
Analog input	6 pins, 10-bit	Up to 18 channels, 12-bit
Extras	6 PWM pins	2 DACs, touch pins, PWM on most pins

✓ Both use the same Arduino IDE and language. Pick Uno to learn, ESP32 for IoT.


ITS TIME TO KNOW
THE BOT

ESP FAMILY

WHAT ARE WE BUILDING?

in other words,
इतनी महाभारत क्यों
सुनी हमने?

PICK-AND-PLACE BOT

Chassis

Motors

Gripper

Gears

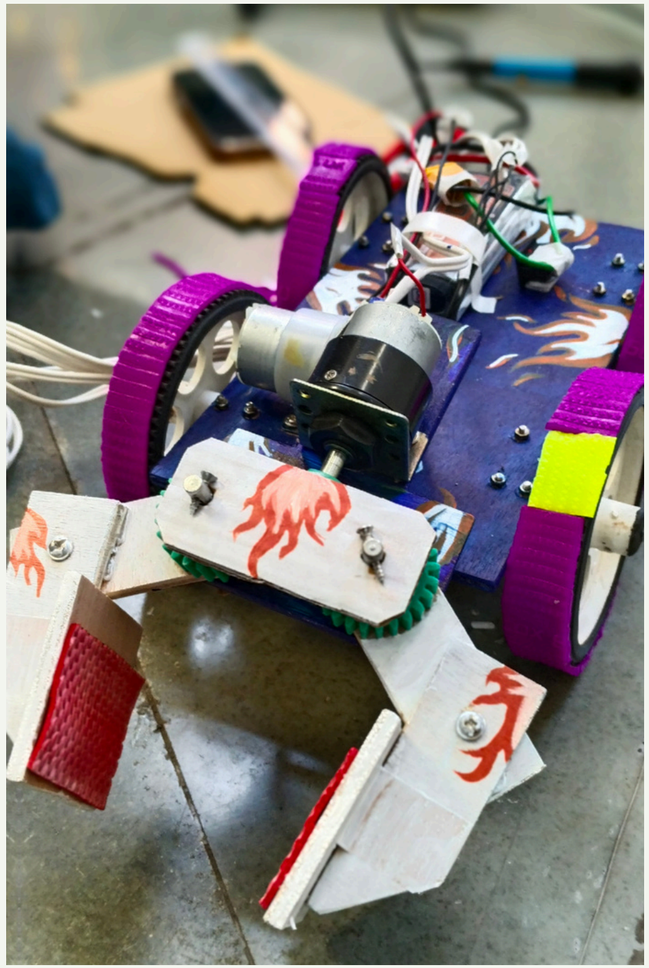
Circuit

Code

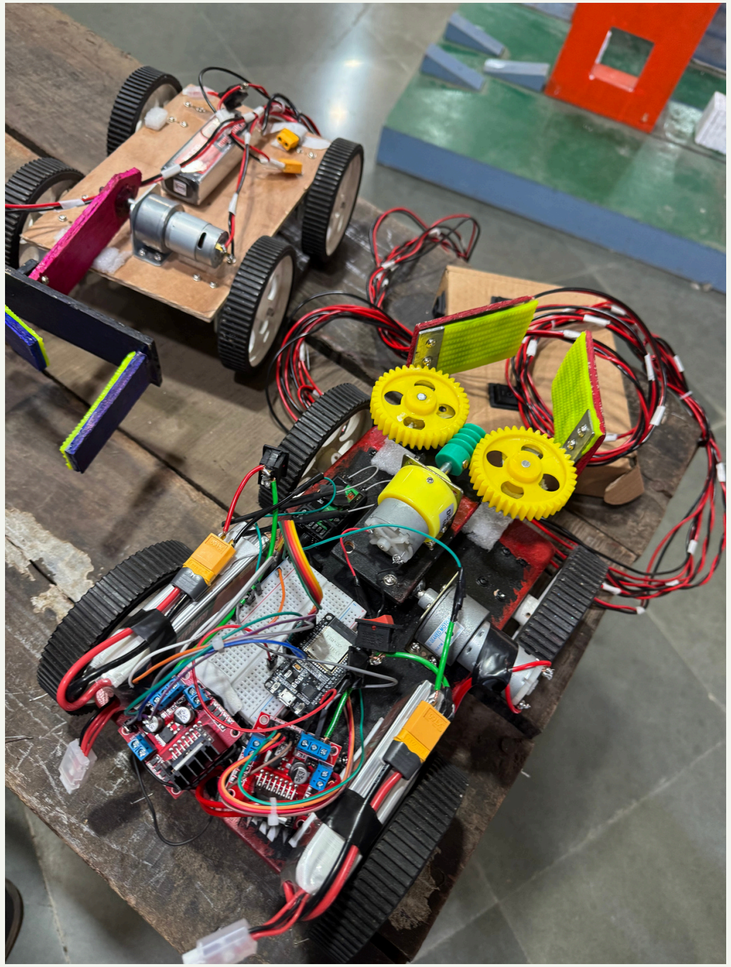
Microcontroller

what else?

WHAT IT LOOKS LIKE...







ACTUATORS

Different types of actuators

An actuator turns a control signal and energy into motion or force. The controller thinks; the actuator acts.

Electric

DC motors, stepper motors, servos, solenoids, relays.

Clean, easy to control. Best for small robots.

Hydraulic

Pressurised liquid pushes a piston.

Very strong: excavators, lifts, presses.

Pneumatic

Compressed air drives a cylinder.

Fast and clean: grippers, factory lines.

Smart materials

Piezo crystals and shape-memory alloys change shape with voltage or heat.

Tiny, precise motion.

In our bot, DC motors and Servos are the actuators used to build the gripper

ACTUATORS

How a DC motor works

1. Current flows through the coil on the rotor (armature).

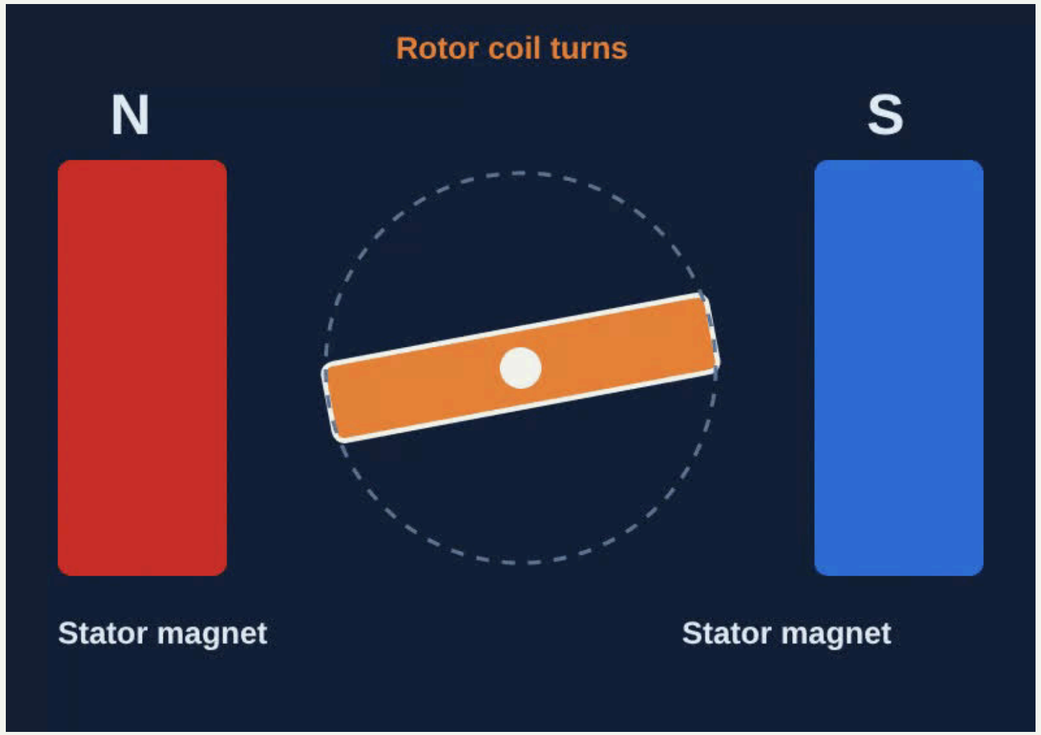
2. The coil sits in the magnetic field of the stator magnets, so each side feels a force in opposite directions.

3. The forces make a torque and the rotor turns.

4. The commutator and brushes flip the current every half turn, so it keeps spinning the same way.

Speed rises with voltage (use PWM).

Torque rises with current.



ACTUATORS

Brushed vs brushless motors

Feature	Brushed	Brushless (BLDC)
Switching	Mechanical: brushes and commutator	Electronic: controller (ESC)
Wear	Brushes wear out and spark	No contact, long life
Efficiency	Lower	Higher
Noise	Noisier	Quieter and smoother
Wiring	2 wires, simple	3 wires plus an ESC
Cost	Low	Higher
Typical use	Toys, small robots	Drones, fans, EVs

ACTUATORS

DC motor direction control

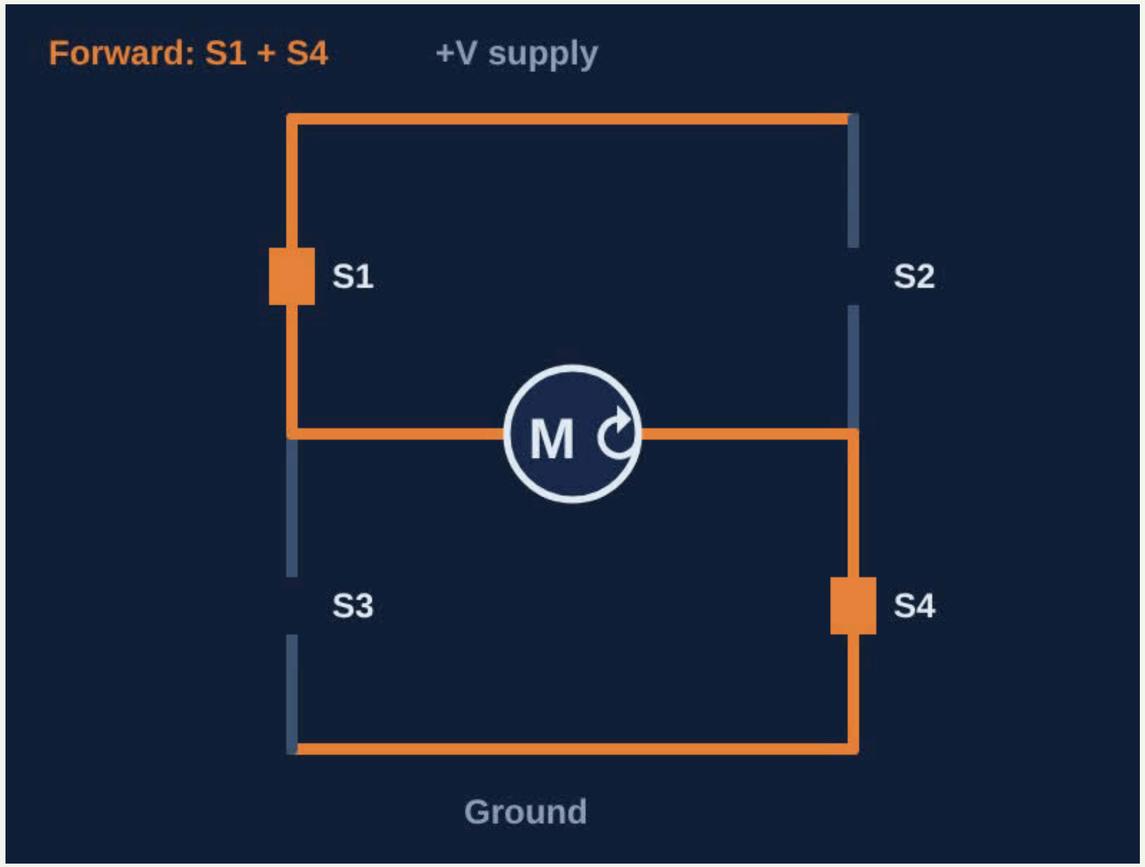
Swap the polarity across a DC motor and it spins the other way. An H-bridge does this with four switches.

Forward: close S1 and S4.

Reverse: close S2 and S3.

Never close S1 and S3 (or S2 and S4) together: that shorts the supply.

Driver chips such as the L298N and L293D contain H-bridges.



ACTUATORS

Driving a motor with an L298N

IN1	IN2	Motor
HIGH	LOW	Forward
LOW	HIGH	Reverse
LOW	LOW	Stop (coast)
HIGH	HIGH	Brake

```
const int IN1=8, IN2=9, ENA=10;
void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENA, OUTPUT);
}
void loop() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW); // forward
  analogWrite(ENA, 200);
  delay(2000);
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH); // reverse
  delay(2000);
}
```

ACTUATORS

Servo motor

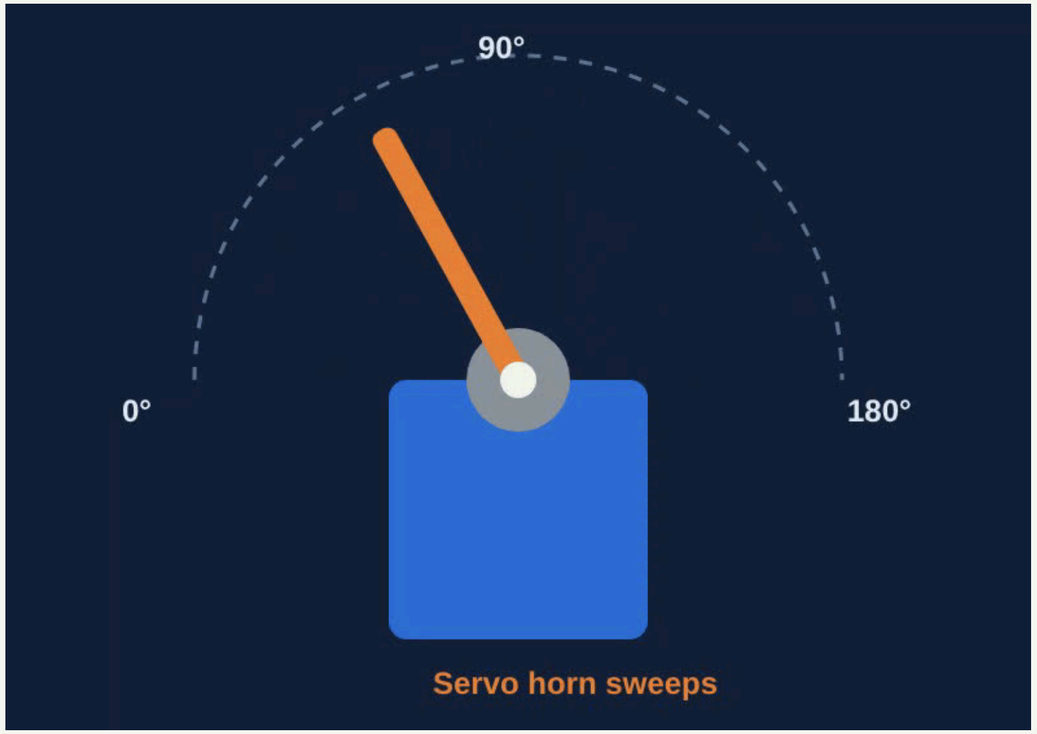
A servo moves to a commanded angle and holds it. Inside is a closed feedback loop:

DC motor provides the motion.

Gearbox trades speed for torque.

Potentiometer reports the shaft angle.

Control circuit compares target with actual and corrects.



GRIPPER

Actuators to build the gripper

Actuators used:

- **DC motor**

Used in the chassis as well as to lift the gripper up/down

- **Servo motor (MG90s)**

Used for the opening and closing of the gripper

Why servo?
We get control over the angle! Other advantages include optimising the weight of the gripper and current draw in the overall circuit.

HOW TO BUILD THE GRIPPER EXACTLY?

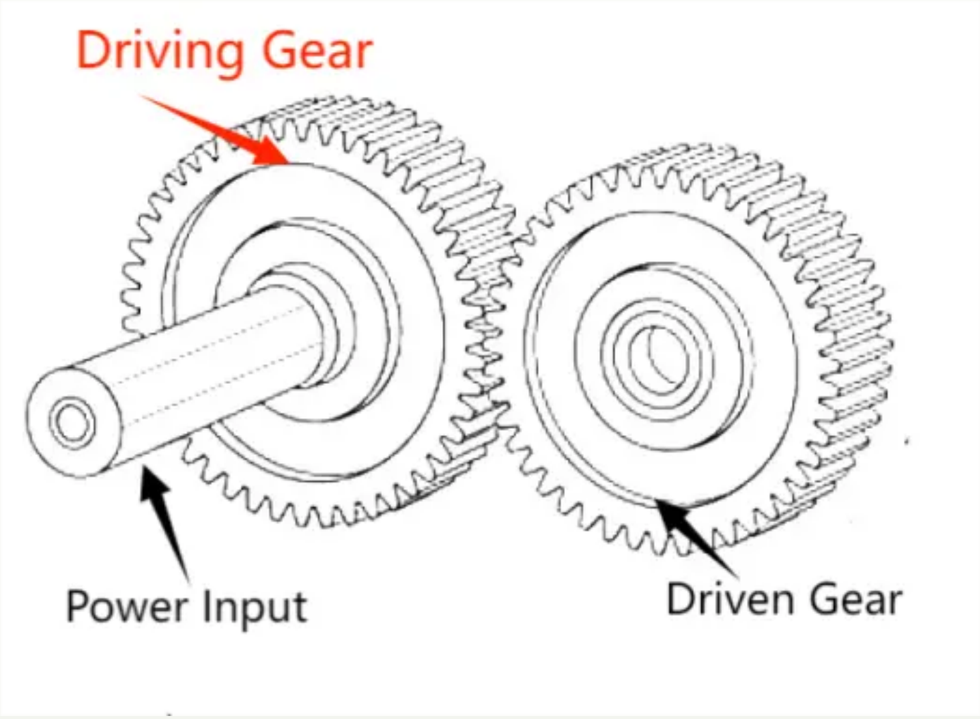
- What kind of gears will be used?
- How much load is the gripper expected to pick up?
- Overall gear mechanism?
- What motors will be used for what?



GRIPPER

A little about gears first!

In any gear arrangement, there is always one driver / driving gear, and the remaining ones are called driven gears.



Gear types



Selection of gear type depends on use-case, for instance, rack and pinion system is useful to convert rotational motion to translation

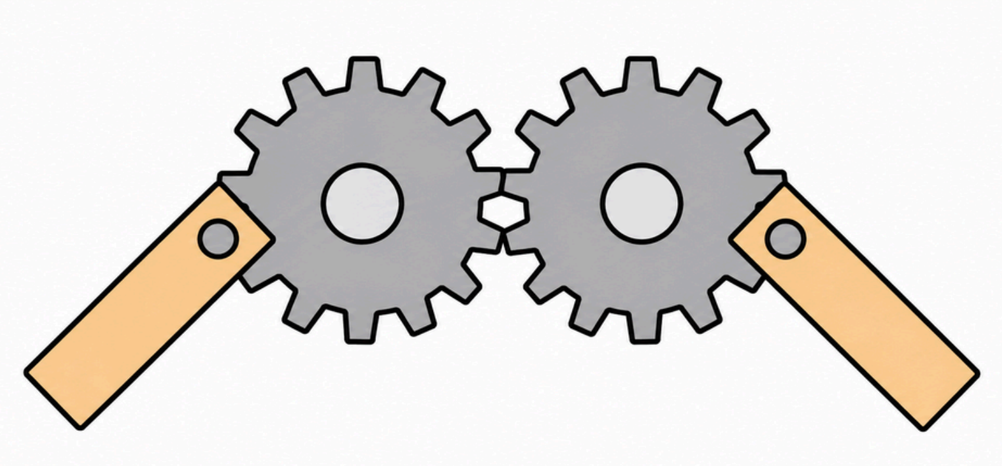
Gear ratio

$$\text{Gear Ratio} = \frac{\text{Teeth on Driven Gear (Output)}}{\text{Teeth on Drive Gear (Input)}}$$

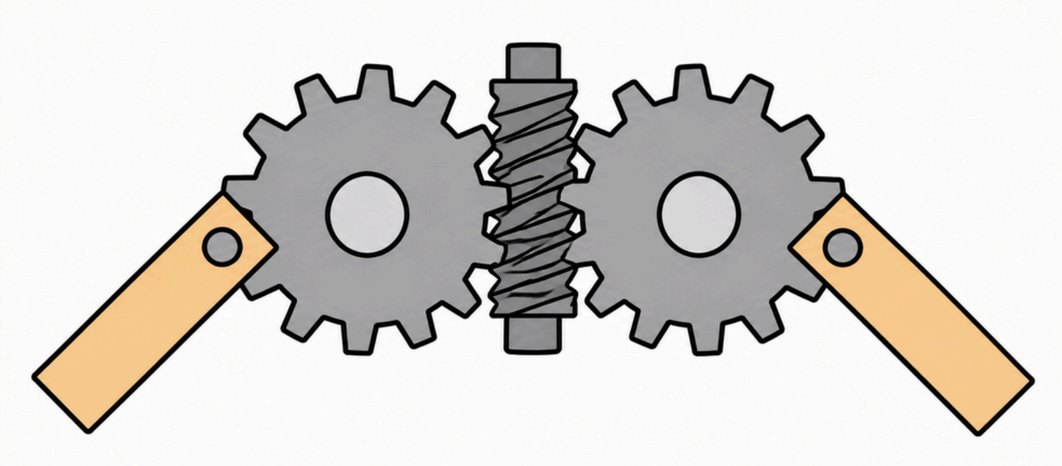
$$\text{Gear Ratio} = \frac{\text{Input RPM } (RPM_{in})}{\text{Output RPM } (RPM_{out})}$$

GRIPPER

Exploring possible gripper mechanisms



Either can be the driving gear, the other then becomes the driven gear



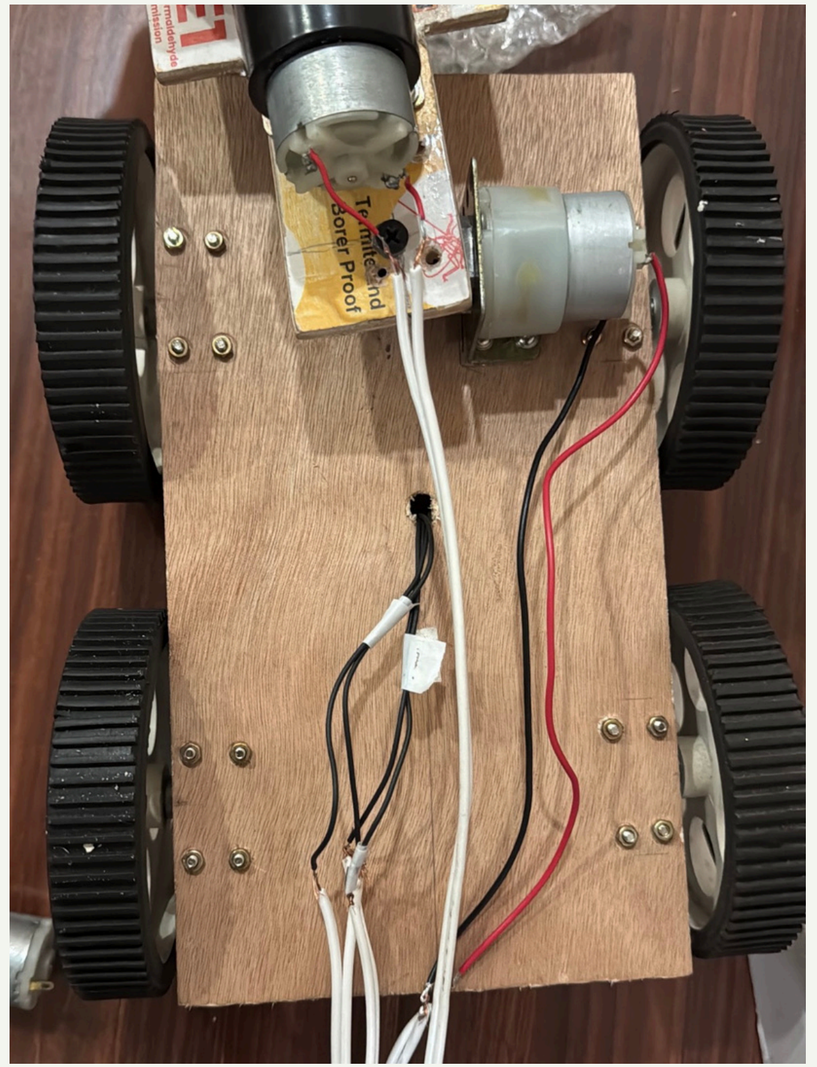
The central worm gear is the driving gear, the others then become driven gears

CHASSIS

Using actuators to build the chassis

Chassis..?
The main body/structure of the bot is called the chassis

Connections:
4 DC motors will be used to drive the bot. To be able to control the movement of the bot in all 4 directions (forward, backward, left, right), motors will be connected in pairs, in parallel



CIRCUIT

Revisiting key parts of any bot

5 MAIN COMPONENTS

- Control Unit / Brain: The controller or processor (such as a microcontroller)
- Sensors (Perception System): Devices like cameras, ultrasonic sensors, gyroscopes that gather data about the robot's internal state and external environment.
- Actuators (Muscles): Electric, hydraulic, or pneumatic motors that convert stored energy into physical movement, rotation, or force.
- Power System: BatteriesMechanical Structure & Frame: The physical body, chassis, joints, arms, and end-effectors (such as grippers or tools)

CIRCUIT

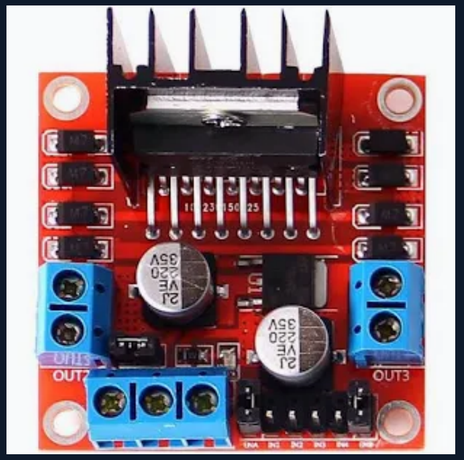
Components used

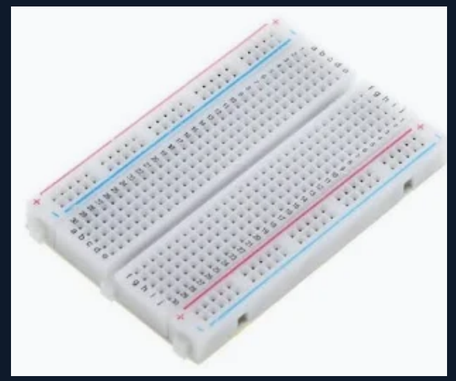
- ESP32
- L298N motor drivers
- DC Motors
- Servo motor
- 3S Li-ion battery
- Breadboard
- Jumper wires

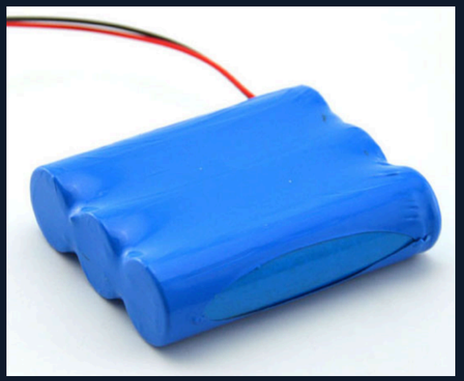
Your phone will be used as the remote!

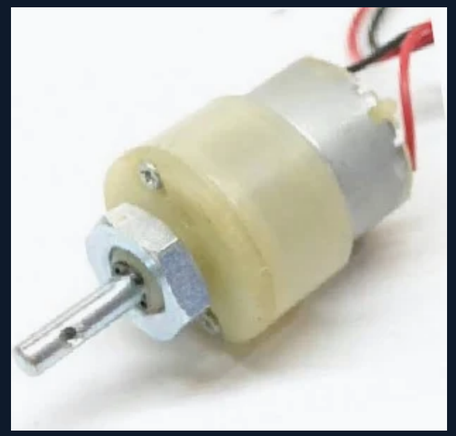
What is the role of each component used here?









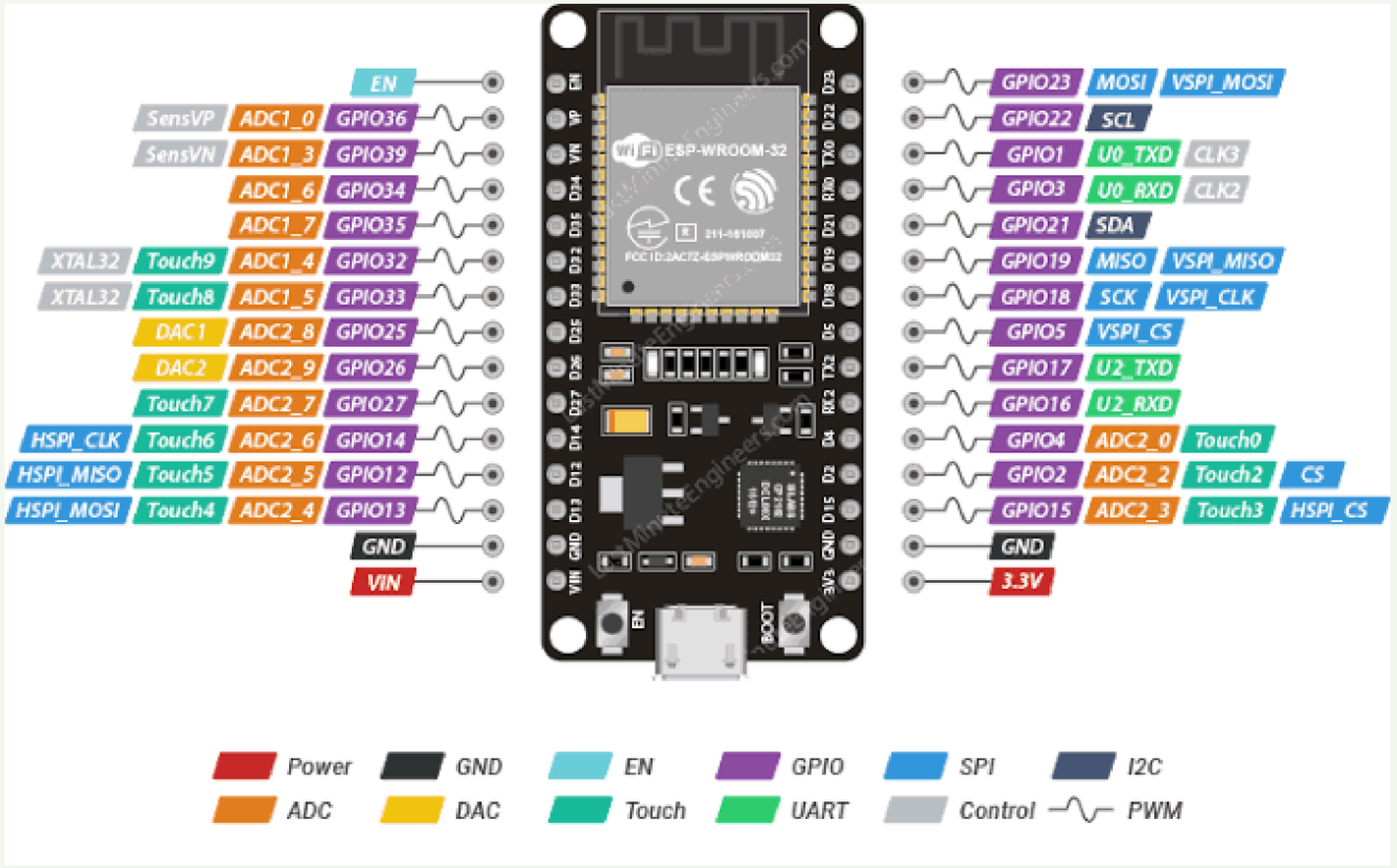




CIRCUIT

Let’s understand the use of each component

ESP32 -
Brain of the bot (Microcontroller)

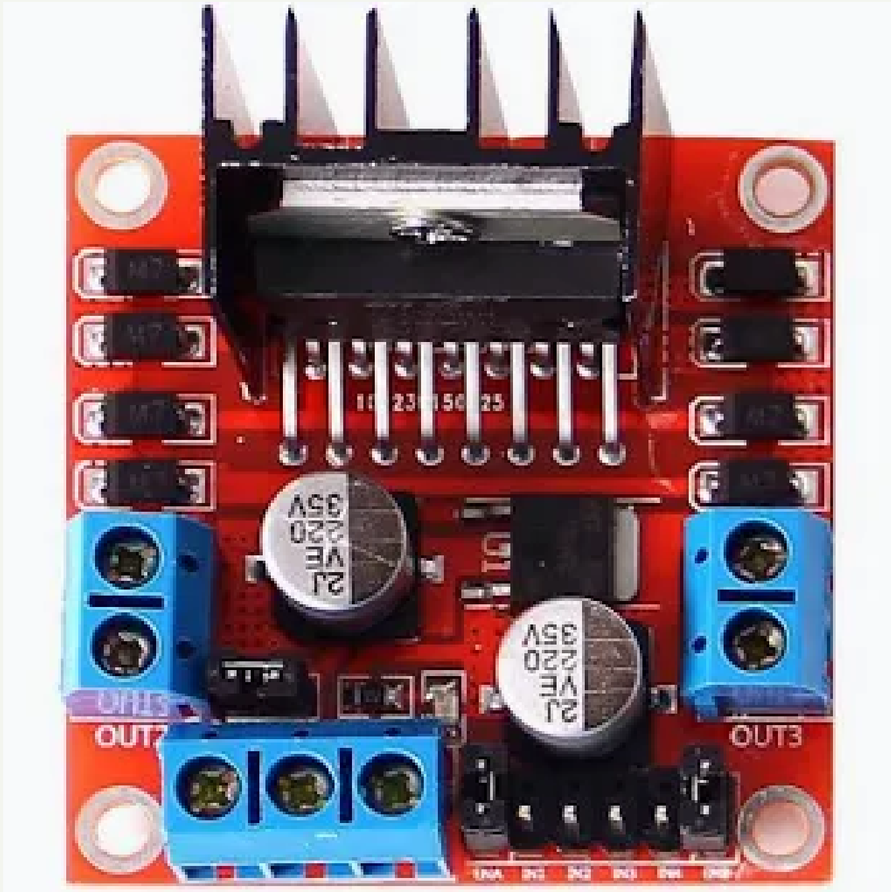


CIRCUIT

Let’s understand the use of each component

L298N - Motor driver

Understanding the pinout:
Power terminals: 12V, GND, 5V
IN pins: IN1-IN3
OUT pins: OUT1-OUT4
ENABLE pins: ENA, ENB



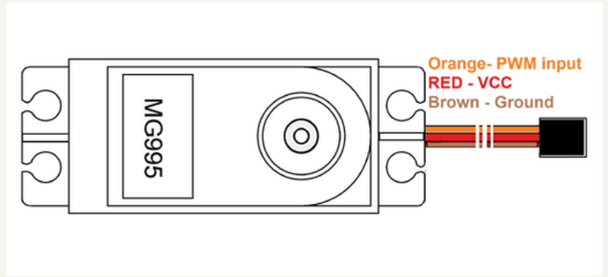
CIRCUIT

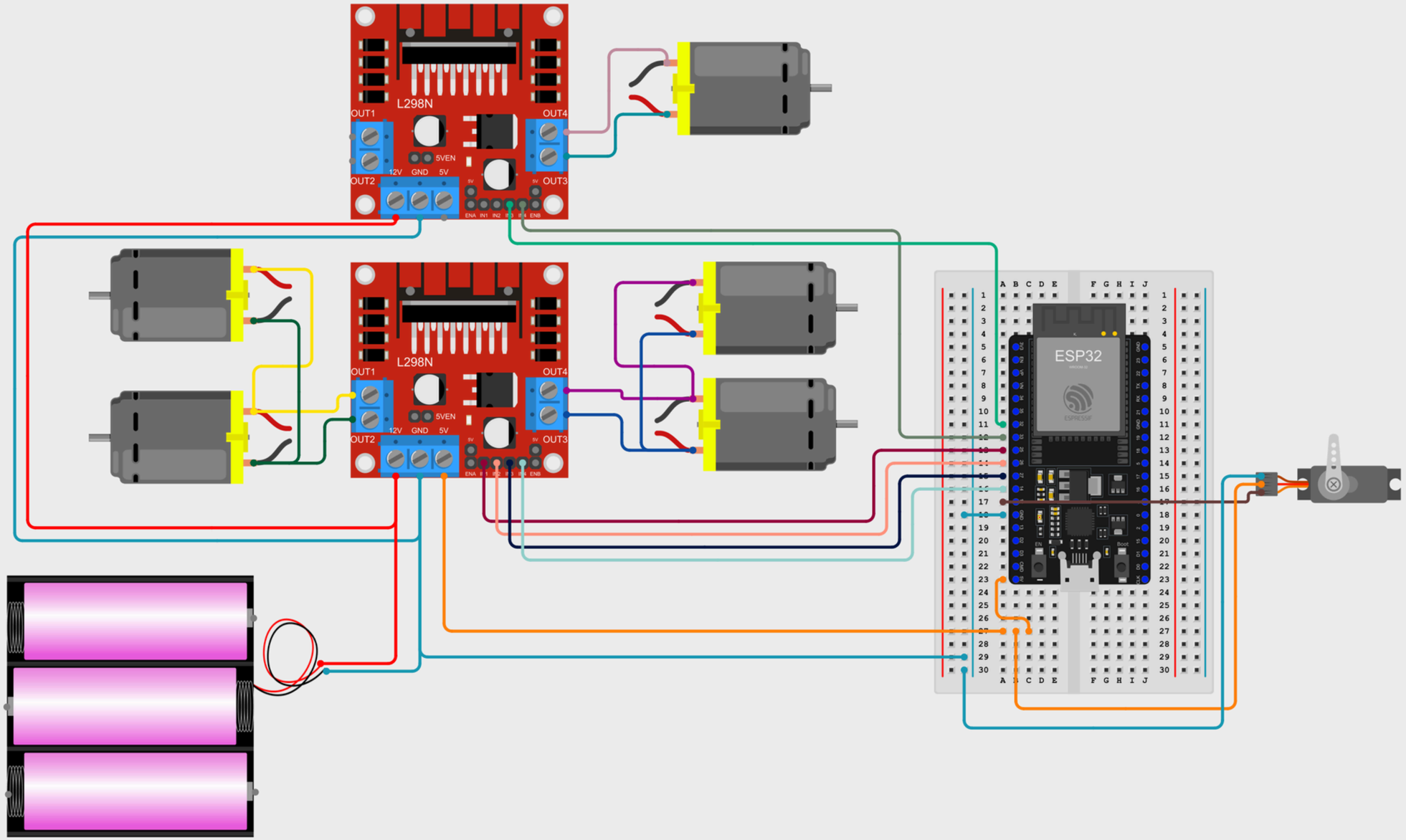
Let’s understand the use of each component

Servo (MG90S) - actuator for gripper

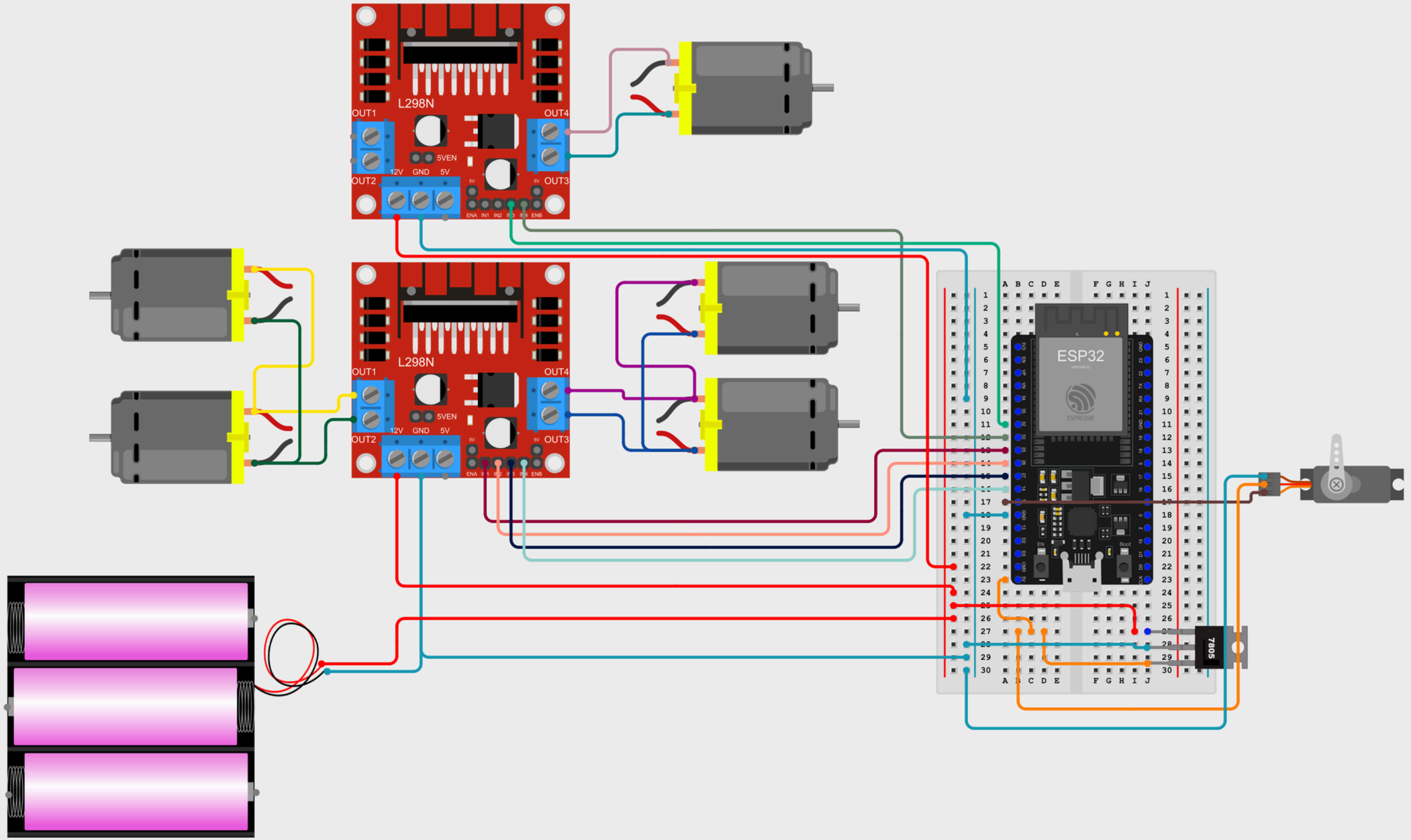
Understanding the pinout:
Red: +ve
Brown: ground
Yellow: signal






Cirkit Designer
L298N
OUT1
OUT2
OUT3
OUT4
5VEN
12V
GND
5V
ENA
IN1
IN2
IN3
IN4
ENB
L298N
OUT1
OUT2
OUT3
OUT4
5VEN
12V
GND
5V
ESP32

Cirkit Designer




menti.com
2196 7764

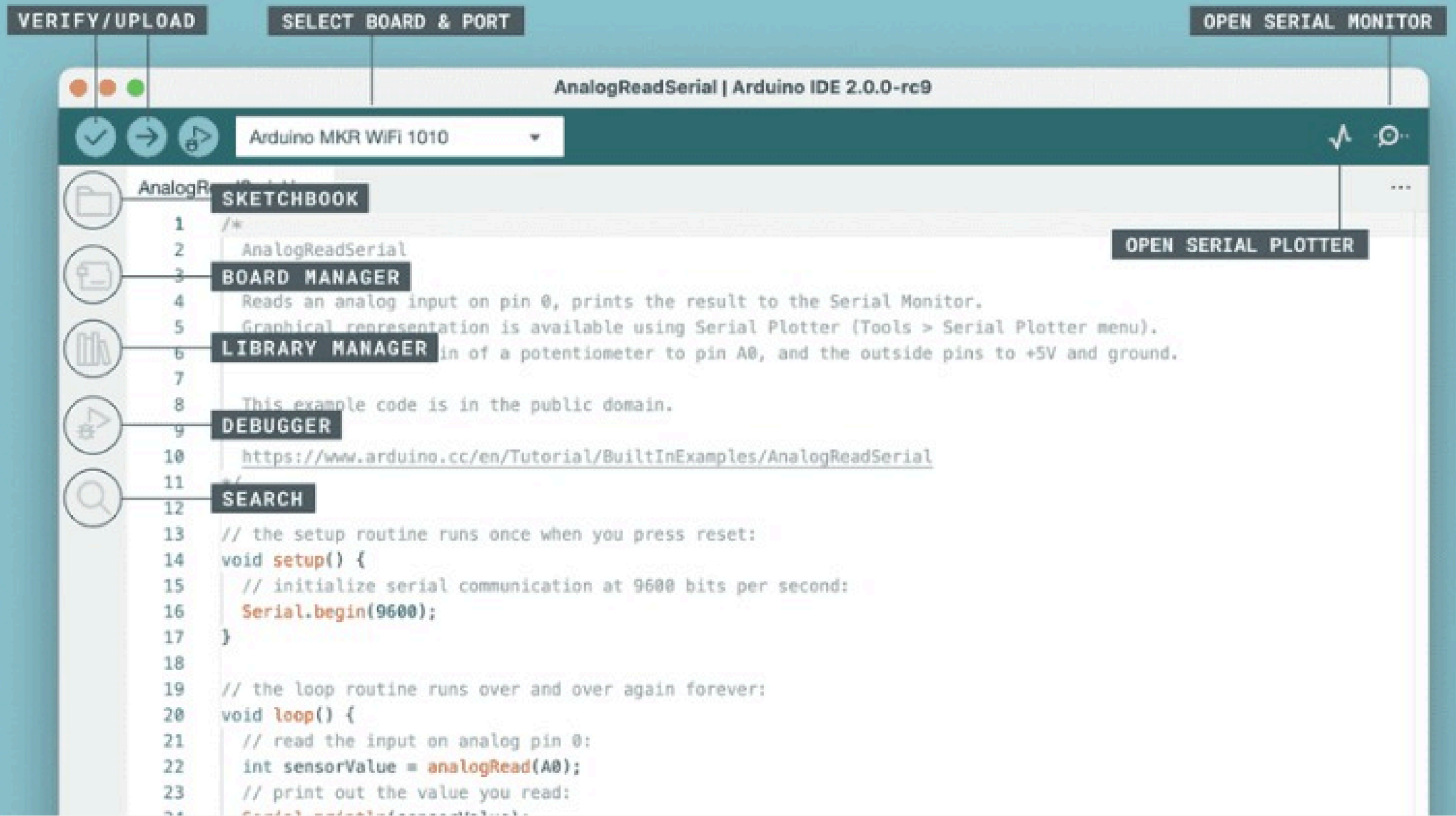
DIFFERENT WAYS TO PROGRAM

A good hardware like ESP32 will be more user friendly if it can be programmed (writing code) in more than one way. And not surprisingly, the ESP32 supports multiple programming environments.

Some of the commonly used programming environments are:

- Arduino IDE
- PlatformIO IDE (VS Code)
- LUA
- MicroPython
- Espressif IDF (IoT Development Framework)
- JavaScript

As Arduino IDE is already a familiar environment, it is better to use the same to program ESP32 in our upcoming project.


VERIFY/UPLOAD
SELECT BOARD & PORT
OPEN SERIAL MONITOR
AnalogReadSerial | Arduino IDE 2.0.0-rc9
Arduino MKR WiFi 1010
SKETCHBOOK
BOARD MANAGER
LIBRARY MANAGER
DEBUGGER
SEARCH
OPEN SERIAL PLOTTER
/*
AnalogReadSerial
Reads an analog input on pin 0, prints the result to the Serial Monitor.
Graphical representation is available using Serial Plotter (Tools > Serial Plotter menu).
in of a potentiometer to pin A0, and the outside pins to +5V and ground.
This example code is in the public domain.
https://www.arduino.cc/en/Tutorial/BuiltInExamples/AnalogReadSerial
*/
// the setup routine runs once when you press reset:
void setup() {
// initialize serial communication at 9600 bits per second:
Serial.begin(9600);
}
// the loop routine runs over and over again forever:
void loop() {
// read the input on analog pin 0:
int sensorValue = analogRead(A0);
// print out the value you read:

PROGRAMMING

Anatomy of a sketch: pinMode and blink

```
// runs once
void setup() {
  pinMode(13, OUTPUT);
}

// runs forever
void loop() {
  digitalWrite(13, HIGH);
  delay(1000);
  digitalWrite(13, LOW);
  delay(1000);
}
```

setup() runs once: configure pins and Serial.

loop() repeats forever.

pinMode(pin, mode): OUTPUT, INPUT or INPUT_PULLUP.

Rules: case-sensitive, statements end with ;, // is a comment.

PROGRAMMING

Functions

```
// returnType name(parameters)
int addTwo(int a, int b) {
  return a + b;
}

void blink(int pin) {
  digitalWrite(pin, HIGH);
  delay(200);
  digitalWrite(pin, LOW);
  delay(200);
}

void loop() {
  blink(13);
  int s = addTwo(3, 4);
}
```

A function is a named block of code you can reuse.

void means it returns nothing.

Parameters pass values in. **return** sends one back.

setup() and loop() are functions too.

PROGRAMMING

Variables and scope

Type	Bytes	Example
boolean	1	true or false
char	1	char c = 'x';
int	2	int temp = 48;
long	4	long t = 5;
float	4	float h = 2.5;

Global: declared outside functions, seen everywhere.
Local: declared inside a function, seen only there.

```
int ledPin = 13; // global

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  int wait = 500; // local
  digitalWrite(ledPin, HIGH);
  delay(wait);
  digitalWrite(ledPin, LOW);
  delay(wait);
}
// wait is not visible in setup()
```

PROGRAMMING

Reading data from the Serial Monitor

```
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
}
void loop() {
  if (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '1') digitalWrite(13, HIGH);
    if (c == '0') digitalWrite(13, LOW);
    Serial.print("Got: ");
    Serial.println(c);
  }
}
```

Serial.begin(9600) sets the baud rate. The monitor must use the same value.

Serial.available() counts bytes waiting.

Serial.read() takes one character. **Serial.parseInt()** reads a number.

print() stays on the line, println() adds a new line.

PROGRAMMING

Operators: comparison and logical

Op	Meaning	Example
==	equal to	12 == 12 is true
!=	not equal	12 != 10 is true
<	less than	12 < 10 is false
>	greater than	12 > 10 is true
<=	at most	12 <= 12 is true
>=	at least	10 >= 12 is false

&& AND

True only if both sides are true.

|| OR

True if at least one side is true.

! NOT

Flips true to false and back.

Watch out: = assigns, == compares. Integer 7 / 2 gives 3.

PROGRAMMING

if, else if and else

```
int temp = 32;

if (temp > 35) {
  Serial.println("Hot");
} else if (temp >= 20 && temp <= 35) {
  Serial.println("Warm");
} else {
  Serial.println("Cool");
}
```

The program checks conditions from top to bottom.

The **first true** branch runs, the rest are skipped.

else catches everything that is left.

With temp = 32 the output is **Warm**.

PROGRAMMING

switch ... case

```
switch (command) {
  case 'F':
    forward();
    break;
  case 'B':
    backward();
    break;
  default:
    stopMotors();
}
```

Compares one variable against many fixed values.

break ends the case. Without it, execution falls through to the next case.

default runs when nothing matches.

Use it for menus and Bluetooth or Serial commands.

PROGRAMMING

Loops: for, while, do-while

for

```
for (int i=0; i<5; i++) {
  blink(13);
}
```

Use when you know how many repeats.

while

```
while (digitalRead(2)==LOW) {
  delay(10);
}
```

Checks first. Runs while the condition is true.

do ... while

```
do {
  x++;
} while (x < 10);
```

Runs at least once, then checks.

⚠ **Avoid endless loops:** something inside must change the condition.

PROGRAMMING

Sensors

A sensor turns a physical quantity into an electrical signal. Analog sensors give a range; digital sensors give on or off.

LDR
Light level. Analog.

LM35 / DHT11
Temperature (and humidity).

HC-SR04
Ultrasonic distance from echo time.

IR sensor
Obstacle or line detection. Digital.

PIR
Human motion. Digital.

Potentiometer
Knob position. Analog.

analogRead(pin) returns 0 to 1023 on an Uno (10-bit).

digitalRead(pin) returns HIGH or LOW.

PROGRAMMING

Project: automatic night light

```
const int ldrPin = A0;
const int ledPin = 9;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}
void loop() {
  int light = analogRead(ldrPin);
  Serial.println(light);
  if (light < 400) digitalWrite(ledPin, HIGH);
  else digitalWrite(ledPin, LOW);
  delay(200);
}
```

Wiring: LDR and a 10 kΩ resistor form a voltage divider into A0.

Logic: dark room gives a low reading, so the LED turns on.

Tune: watch the Serial Monitor and adjust 400.

Uses Serial, analogRead, if-else, pinMode and variables.

BUILD: PICK-AND-PLACE ROBOT

Our robot and its code

A phone sends one-letter commands over Bluetooth. The ESP32 reads each letter and switches pins that drive the motors and the gripper.

Phone app
Sends letters such as f, b, l, r

→

ESP32
Runs our code and sets pins HIGH or LOW

→

Motor driver
H-bridge boards for the DC motors

→

Actuators
Wheels, lift motor, gripper servo

We will code: forward, backward, turns, stop, lift and gripper.

Provided for you: the Bluetooth (BLE) setup that receives the letters.

Our robot and its code

BUILD: PICK-AND-PLACE ROBOT

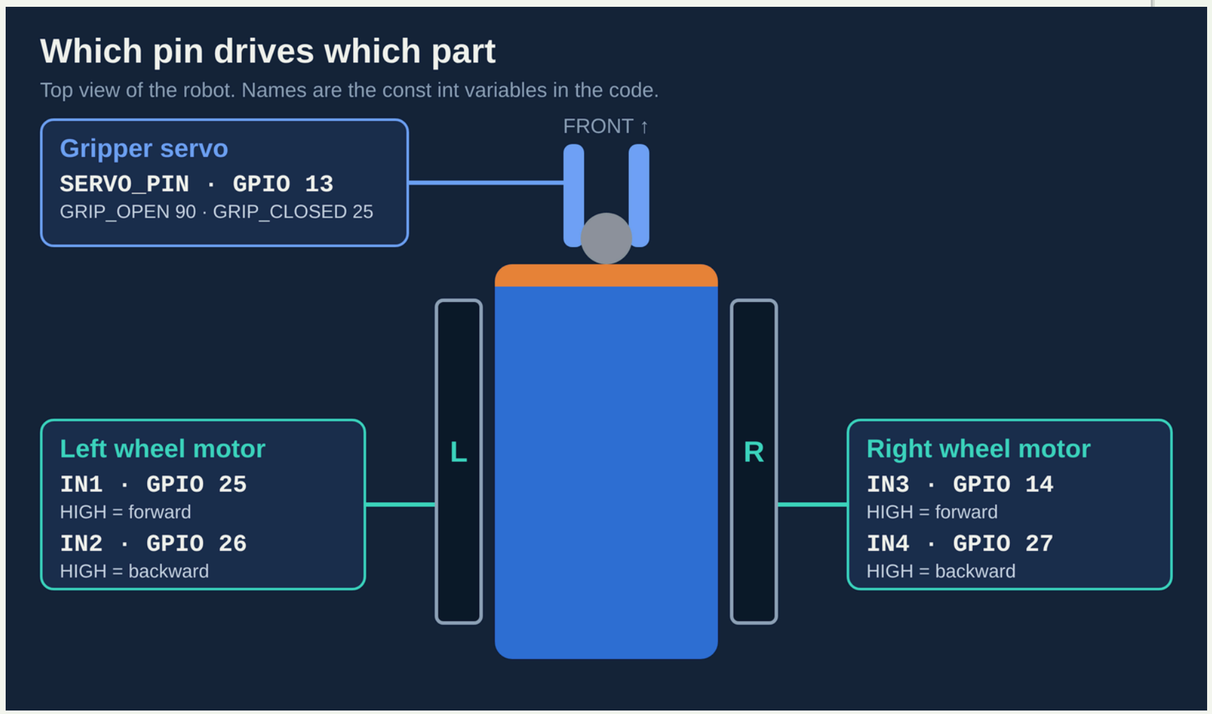
Step 1: name the pins

```
const int IN1 = 25;
const int IN2 = 26;
const int IN3 = 14;
const int IN4 = 27;

void setup() {
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
}
```

Note: the numbers are ESP32 GPIO numbers, not positions on the board.

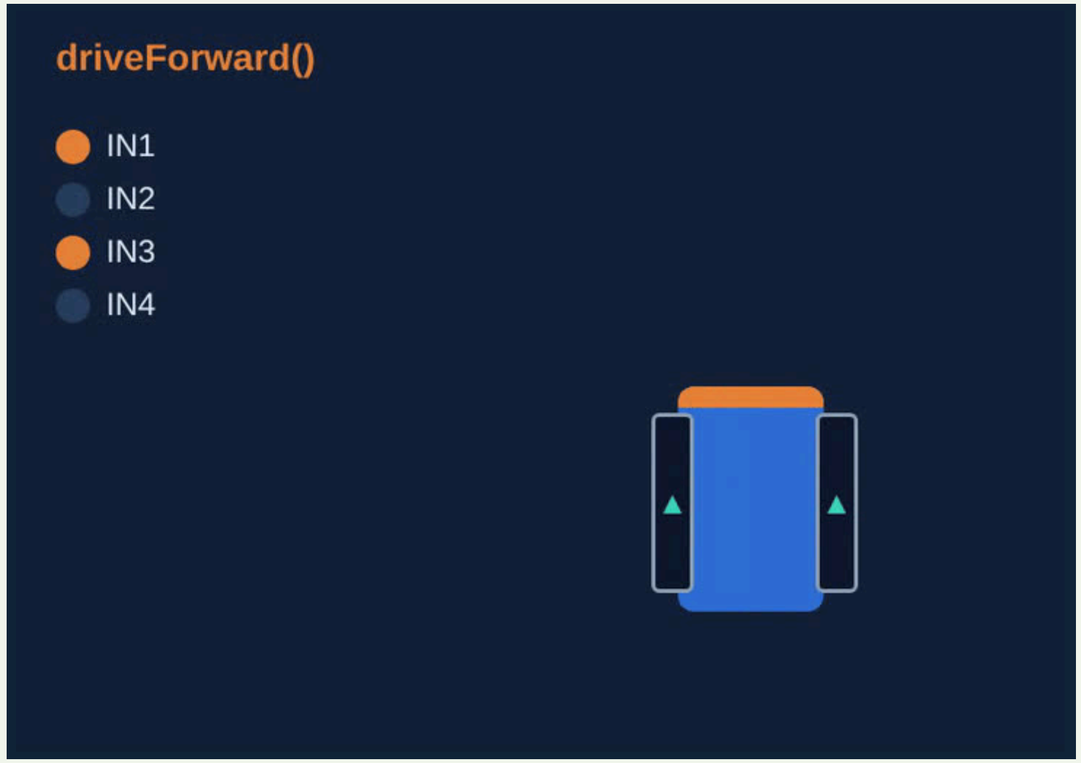
GPIO	Name	Job
25,26	IN1, IN2	Left wheel motor
14, 27	IN3, IN4	Right wheel motor
13	SERVO_PIN	Gripper Servo



Step 2: forward and backward

```
void driveForward() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}
void driveBackward() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}
```

Rule: HIGH on IN1 and IN3 spins both wheels forward. HIGH on IN2 and IN4 spins both backward.



Step 3: turning left and right

```
void turnLeft() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, HIGH);
  digitalWrite(IN3, HIGH);
  digitalWrite(IN4, LOW);
}
void turnRight() {
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, HIGH);
}
```

Idea: the two wheels spin in opposite directions, so the robot pivots on the spot.



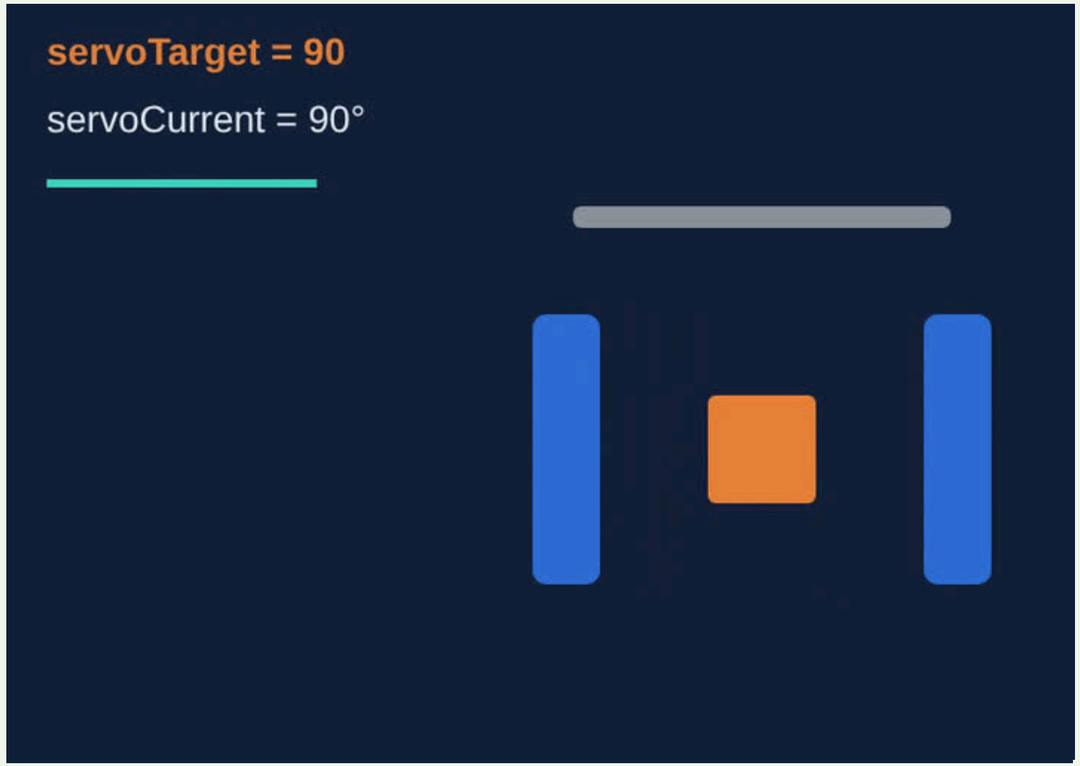
Step 4: the gripper servo

```
const int GRIP_OPEN = 90;
const int GRIP_CLOSED = 25;
int servoCurrent = GRIP_OPEN;
int servoTarget = GRIP_OPEN;

// 'o' opens, 'c' closes
servoTarget = GRIP_OPEN;
servoTarget = GRIP_CLOSED;

// every 15 ms in loop()
if (servoCurrent < servoTarget) servoCurrent++;
else if (servoCurrent > servoTarget) servoCurrent--;
gripper.write(servoCurrent);
```

Smooth motion: the code moves one degree every 15 ms toward the target instead of jumping.



Step 5: letters become actions

```
void processCommand(char command) {
  switch (command) {
    case 'f': driveForward(); break;
    case 'b': driveBackward(); break;
    case 'l': turnLeft(); break;
    case 'r': turnRight(); break;
    case 's': stopDrive(); break;
    case 'u': liftUp(); break;
    case 'd': liftDown(); break;
    case 'x': stopLift(); break;
  }
}
// simplified: o and c set the gripper
```

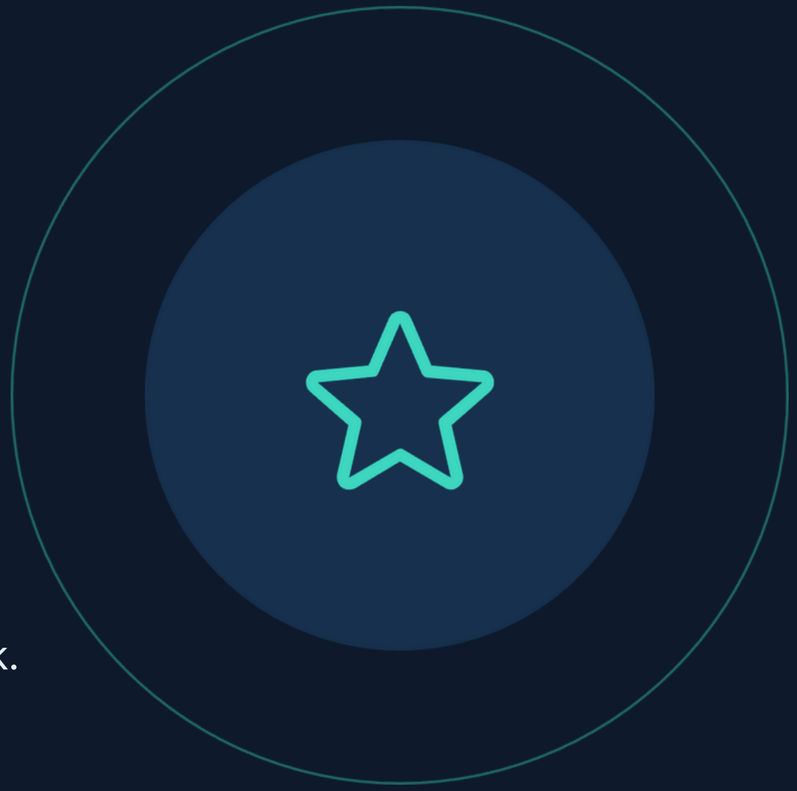
Letter	Action	Code
f	Forward	driveForward()
b	Backward	driveBackward()
l	Turn left	turnLeft()
r	Turn right	turnRight()
s	Stop wheels	stopDrive()
u	Lift up	liftUp()
d	Lift down	liftDown()
x	Stop lift	stopLift()
o	Open gripper	GRIP_OPEN
c	Close gripper	GRIP_CLOSED

WRAP-UP

Sense. Think. Act.

Sensors read the world, the microcontroller decides, and actuators move. You now have the circuit basics, the ESP32 and Arduino boards, motors and servos, and the core code.

Next step: build the night light, then add a servo that turns when it gets dark.



Questions and feedback?